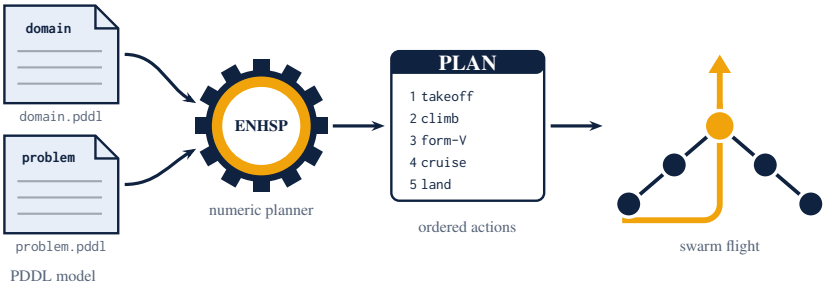
DEPARTMENT OF COMPUTER SCIENCE AND
ENGINEERING
INDIAN INSTITUTE OF TECHNOLOGY MADRAS
CHENNAI – 600036

PDDL-Based Mission Planning for a Drone Swarm Simulator



A Thesis

Submitted by

Vadithe Sai Kumar Naik

For the award of the degree

Of

MASTER OF TECHNOLOGY

December 2026

Thesis Certificate

This is to certify that the thesis entitled **PDDL-Based Mission Planning for a Drone Swarm Simulator**, submitted by **Vadithe Sai Kumar Naik (CS25M053)** to the Indian Institute of Technology Madras in partial fulfilment of the requirements for the degree of **Master of Technology**, is a bona fide record of the work carried out under the guidance of **Prof. N. S. Narayanaswamy**, Project Advisor, and **Dr. Jaimandeep Singh**, Project Co-Advisor.

The contents of this thesis, in full or in part, have not been submitted to any other institute or university for the award of a degree or diploma.

Chennai 600036
Date: December 2026

Vadithe Sai Kumar Naik
CS25M053

Prof. N. S. Narayanaswamy
Project Advisor, Professor
Department of Computer Science and Engineering
Indian Institute of Technology Madras

Dr. Jaimandeep Singh
Project Co-Advisor
Department of Computer Science and Engineering
Indian Institute of Technology Madras

© 2026 Indian Institute of Technology Madras

Acknowledgements

I would like to express my sincere gratitude to my project guide **Prof. N. S. Narayanaswamy** for his exceptional guidance, unwavering support, and encouragement throughout this project. His insights and constructive feedback were instrumental in shaping the direction of this work and continually motivated me to improve its quality.

I am also grateful to my co-guide **Dr. Jaimandeep Singh** for his invaluable support and assistance. His technical insights and thoughtful suggestions broadened my perspective and significantly enhanced the outcome of this project.

Furthermore, I would like to thank the faculty and staff of the Department of Computer Science and Engineering, IIT Madras, for providing the resources and environment necessary for carrying out this work.

Abstract

Keywords: automated planning; PDDL; numeric planning; ENHSP; drone swarm; formation flight; fault handling; battery management.

A drone swarm mission is more than a route. Before a drone may take off, join a formation or continue a leg, its battery must cover the next move, its GPS and radio links must work, its health must be acceptable and its neighbours in the formation must be in place. This thesis describes the planning layer of a drone swarm simulator, in which these rules are written in the Planning Domain Definition Language (PDDL) and solved by the Expressive Numeric Heuristic Search Planner (ENHSP). In this work I designed and built this complete PDDL planning layer.

Ten mission plans were developed, each as a domain file and a problem template: point-to-point travel, forest area search, V formation, two-dimensional grid formation, three-dimensional (cube) grid formation, parabola formation, figure-eight train formation, star drone show, planetary orbit show and road inspection. Every plan shares one vocabulary of drone states, health flags, failure states and numeric fluents for battery, energy, altitude and health. A plan service retargets each template onto the points the operator picks on the map, expands it to the number of selected drones and passes it to ENHSP. Only the leader corridor is usually planned symbolically; the other drones' routes are then derived geometrically from their formation slots.

Inter-drone communication is modelled in two layers. In PDDL, each formation declares which drones are neighbours and every formation action checks a per-drone `neighbor-comm-ok` flag; the parabola plan goes further and requires each directed link to be established within a communication range. At run time the ground station talks to each drone over a JSON-over-UDP protocol with heartbeats, and formation keeping for trains and shows runs on board each drone against a shared, time-stamped pace. Four fault types are modelled in the domains – GPS loss, communication loss, neighbour-link loss and health failure – together with ground-controller responses such as return, hover and sacrifice. Battery health is handled both as a planning resource and at run time through low and critical thresholds that pause the swarm and ask the operator to decide.

Every plan works with a dynamic number of drones: the operator chooses how many drones to use, and the template is expanded to exactly that number before planning. Within the simulator's planning time limit, point-to-point travel adapts to as many as 60 drones, forest search to 7, the V formation to 9 (odd counts), the two-dimensional grid to 36 and the cube formation to 64 drones. The figure-eight train adapts up to the capacity of the eight the operator draws (27 drones on a typical eight), the star show to 6–40 drones and the planetary show to 2–8 drones, while the parabola uses exactly three drones and road inspection one. The report also records the modelling lessons learnt during verification and the limits of the present approach.

Contents

Page

ACKNOWLEDGEMENTS i

ABSTRACT ii

CONTENTS iii

LIST OF TABLES vi

LIST OF FIGURES vii

ABBREVIATIONS ix

CHAPTER 1 INTRODUCTION 1

1.1 Background and Motivation 1
1.2 Problem Statement and Objectives 1
1.3 Scope and Contributions 2
1.4 Organisation of the Report 2

CHAPTER 2 BACKGROUND AND LITERATURE REVIEW 4

2.1 Automated Planning 4
2.2 The Planning Domain Definition Language 4
2.3 Numeric Planning with ENHSP 5
2.4 Multi-Drone Missions and Formations 5
2.5 Fault-Aware Planning for UAVs 5
2.6 Position of This Work 5

CHAPTER 3 SYSTEM ARCHITECTURE AND PLANNING PIPELINE 6

3.1 Simulator Overview 6
3.2 From Map Clicks to Flying Drones 6
3.3 Plan Kinds and Folders 8
3.4 Run Records 8

CHAPTER 4 PDDL MODELLING APPROACH 9

4.1 A Shared Vocabulary 9
4.2 The Common Mission Life Cycle 10
4.3 Design Patterns for Tractable Swarm Models 11

CHAPTER 5 THE MISSION PLANS 13

5.1 Point-to-Point Travel . . . 13
5.2 Forest Area Search . . . 16
5.3 V Formation . . . 18
5.4 Two-Dimensional Grid Formation . . . 20
5.5 Three-Dimensional Cube Formation . . . 22
5.6 Parabola Formation . . . 24
5.7 Figure-Eight Train Formation . . . 26
5.8 Star Drone Show . . . 28
5.9 Planetary Orbit Show . . . 30
5.10 Road Inspection . . . 32
5.11 Summary of the Plans . . . 34

CHAPTER 6 INTER-DRONE COMMUNICATION 35

6.1 Communication in the PDDL Models . . . 35
6.2 The Range-Checked Link Model of the Parabola Plan . . . 36
6.3 Run-Time Communication between Ground Station and Drones . . . 36
6.4 On-Board Formation Keeping . . . 37
6.5 How the Two Levels Relate . . . 38

CHAPTER 7 FAULT MODELLING AND BATTERY HEALTH 40

7.1 Faults in Simple Words . . . 40
7.2 How the Planner Models a Fault . . . 41
7.3 What the Ground Controller Can Order . . . 41
7.4 Battery as a Planning Resource . . . 42
7.5 Battery Health at Run Time . . . 42
7.6 Fault Injection and Operator Decisions . . . 43

CHAPTER 8 VERIFICATION AND RESULTS 44

8.1 Method . . . 44
8.2 In-App Planning Requests . . . 44
8.3 Scalability of the Cube Formation . . . 44
8.4 Figure-Eight Capacity and Flight Quality . . . 45
8.5 Other Plans . . . 45
8.6 A Complete Sample Plan . . . 46
8.7 Defects Found and Fixed . . . 47

CHAPTER 9 DISCUSSION AND LIMITATIONS 48

9.1 What the Models Guarantee . . . 48
9.2 Limitations . . . 48

CHAPTER 10 CONCLUSION AND FUTURE WORK **50**

10.1 Conclusion 50

10.2 Future Work 50

REFERENCES **52**

List of Tables

Table	Caption	Page
3.1	Components of the simulator and their role in planning	6
3.2	Inventory of the ten mission plans	8
4.1	Shared predicates and numeric functions used across the domains	9
5.1	Comparison of the mission plans.	34
6.1	Neighbour relations declared by each formation.	35
6.2	Commands of the ground-station-to-drone protocol	37
7.1	Faults handled in this work, in simple words	40
7.2	Fault detection actions shared by the domains.	41
7.3	Each fault at planning time and during flight	43
8.1	Logged planning requests per plan	44
8.2	Cube formation: ENHSP results through the real pipeline	45
8.3	Figure-eight train: ENHSP results on an eight of capacity 27	45
8.4	Verification summary of the remaining plans	46
8.5	Modelling defects found during verification	47

List of Figures

Figure	Caption	Page
3.1	The planning pipeline from map input to flight	8
4.1	The life cycle shared by the formation plans	11
5.1	Symbols used in the communication diagrams	13
5.2	Communication in point-to-point travel	14
5.3	Point-to-point travel in the simulator	16
5.4	Communication in forest area search	17
5.5	Forest area search in the simulator	18
5.6	Communication in the V formation	18
5.7	V formation in the simulator	20
5.8	Communication in the grid formation	20
5.9	Rigid formations: V, grid and cube	22
5.10	Grid formation in the simulator	22
5.11	Communication in the cube formation	23
5.12	Cube formation in the simulator	24
5.13	Communication in the parabola formation	25
5.14	The parabola formation seen from behind	26
5.15	Parabola formation in the simulator	26
5.16	Communication in the figure-eight train	27
5.17	The figure-eight train	28
5.18	Figure-eight train in the simulator	28
5.19	Communication in the star show	29
5.20	Star show in the simulator	30
5.21	Communication in the planetary show	31
5.22	Shape-changing shows: star and planets	32
5.23	Planetary show in the simulator	32
5.24	Communication in road inspection	33
5.25	Road inspection in the simulator	34
6.1	Communication topologies	35
6.2	Run-time messaging between GCS and drones	36
6.3	On-board formation keeping	38

6.4 Planning-time and run-time communication 39
7.1 Ground-controller responses to a fault 42
7.2 Battery thresholds at planning time and run time 43
8.1 Cube formation scalability. 45

Abbreviations

Abbreviation	Meaning
PDDL	Planning Domain Definition Language
ENHSP	Expressive Numeric Heuristic Search Planner
GCS	Ground Control Station
GUI	Graphical User Interface
UAV	Unmanned Aerial Vehicle
SYSID	MAVLink system identifier of a drone
MAVLink	Micro Air Vehicle Link, a lightweight drone messaging protocol
UDP	User Datagram Protocol
JSON	JavaScript Object Notation
OSRM	Open Source Routing Machine
AIBR	Additive Interval Based Relaxation (ENHSP preprocessing)
RTL	Return to Launch

Chapter 1

Introduction

1.1 Background and Motivation

Small unmanned aerial vehicles are now used for search, inspection, mapping and public light shows. Many of these tasks are done better by a group of drones than by one drone. A group, however, needs coordination. Each drone must know when to take off, which place in the formation it must hold, which other drones it must stay in contact with, and what to do when something goes wrong. Writing these rules directly in flight code makes them hard to read, hard to check and hard to change.

Automated planning offers a cleaner way. In planning, the rules of the world are written once as a *domain*: a set of actions, each with the conditions under which it may be applied and the changes it causes. A particular mission is written as a *problem*: the drones, the places, the starting state and the goal. A general-purpose planner then searches for a sequence of actions that reaches the goal while respecting every rule. PDDL is the standard language for writing such domains and problems.

This project is part of a larger drone swarm simulator built in the department. The simulator runs each drone as an independent thread and offers a ground station GUI with a map, telemetry and fault injection. The work reported here is the planning layer of that simulator: the PDDL models for every mission type, and their link to the rest of the system.

1.2 Problem Statement and Objectives

The central problem is to express swarm missions – including formations, communication requirements, battery limits and failures – in PDDL in a way that a numeric planner can solve quickly for realistic swarm sizes, and whose plans can be turned into flyable routes. The work had the following objectives:

1. Design a common PDDL vocabulary for drone state, health, communication, battery and failure that every mission plan can share.
2. Build domain and problem files for a range of missions: travel, area search, rigid formations (V, grid, cube, parabola), path-following trains (figure-eight), shape-changing shows (star, planets) and road inspection.
3. Model drone-to-drone communication, so that a formation can only form and fly while the required links are up.

4. Model faults (GPS, communication, neighbour link, health) and battery health, together with the ground controller's responses.
5. Integrate the plans with the simulator, so that a plan is retargeted to the operator's map inputs, solved by ENHSP and executed by the drones, and verify the results.

1.3 Scope and Contributions

My main contribution in this work is the complete PDDL planning layer of the simulator. In particular, this work contributes:

- ten mission plans (Table 3.2), about 6 700 lines of PDDL in total, written on one shared vocabulary of predicates and numeric fluents, each of which adapts to the number of drones the operator selects;
- a set of modelling patterns that keep formation problems tractable for large swarms: planning only the leader corridor, descriptive neighbour relations with per-drone communication flags, and propositional rather than numeric gates for ordered launches;
- a two-level model of communication: symbolic neighbour links in PDDL, including an explicit range-checked link model in the parabola plan, and the run-time protocol used by the ground station and the drones;
- a fault and battery model covering four failure types, ground-controller responses and energy accounting for travel, climb, cruise and hover;
- verification of every plan through the real planning pipeline, including scalability measurements and a record of the modelling defects found and fixed.

Besides the planning layer, I also worked on parts of the ground station GUI, in particular the mission planner panel through which the operator chooses a plan and picks its points on the map. The remaining parts of the simulator, such as the MAVLink message conversion and the flight firmware emulation, were developed by other members of the project team. This thesis therefore focuses on PDDL-based planning; the other parts are described only as far as needed to explain how a PDDL plan becomes a flight. All results in this report come from the simulator.

1.4 Organisation of the Report

Chapter 2 reviews automated planning, PDDL, numeric planning and related work on multi-drone missions. Chapter 3 explains the simulator architecture and the planning pipeline step by step. Chapter 4 presents the common PDDL vocabulary and design patterns. Chapter 5 explains all ten mission plans one by one: what each mission does, how its drones communicate, the important parts of its PDDL and its output. Chapter 6 explains how drones communicate, both in the model and at run time.

Chapter 7 covers faults and battery health. Chapter 8 reports the verification results and the defects found. Chapter 9 discusses limitations, and Chapter 10 concludes with future work.

Chapter 2

Background and Literature Review

2.1 Automated Planning

Classical planning studies how an agent can reach a goal from a known initial state by applying actions with known effects. A planning task is described by a set of state variables, a set of actions and a goal. Each action has a *precondition* that must hold before it is applied and an *effect* that changes the state. A *plan* is a sequence of actions that, applied in order from the initial state, ends in a state that satisfies the goal. Heuristic search planners find such plans by estimating, for each state, how far it is from the goal and expanding the most promising states first.

2.2 The Planning Domain Definition Language

PDDL was introduced to give planning competitions a common input language. A PDDL model is split into two files:

- the **domain** defines object types, *predicates* (true/false facts such as `(gps-ok ?d)`), *functions* (numeric values such as `(battery ?d)`) and *actions*;
- the **problem** names the concrete objects, gives the initial facts and values, and states the goal.

PDDL 2.1 added *numeric fluents*, so that actions can test and change numbers. For example, a move can require enough charge and then use it up:

```
:precondition (> (battery ?d) (energy-required ?from ?to))
:effect       (decrease (battery ?d) (energy-required ?from ?to))
```

Later PDDL versions added further requirements. The three used most in this work are:

- `:universal-preconditions` – a condition over every object of a type, for example "*every* wing drone is in its slot";
- `:conditional-effects` – an effect that applies only to some objects, for example "move each drone *that is* a wing drone";
- `:negative-preconditions` – a condition that a fact is *false*, for example "only if this drone is *not* already in formation".

2.3 Numeric Planning with ENHSP

ENHSP is a heuristic search planner for PDDL 2.1 problems with numeric fluents. It grounds the domain into concrete actions, runs a relaxation-based preprocessing step (AIBR) that also detects some unsolvable problems early, and then searches with numeric heuristics. Two properties of ENHSP mattered in this project. First, its heuristics estimate progress well for conditions that compare a fluent with a constant, but poorly for conditions that compare two fluents with each other. Second, its grounding step can use a large amount of memory for domains with many drones and quantified conditions; a "naive" grounder option (`-gro naive`) helps for formation domains. Both points shaped the models in Chapter 4.

2.4 Multi-Drone Missions and Formations

Aerial swarm research covers formation control, task allocation and collective behaviour. Formation flight is often treated with a leader–follower scheme, in which one drone follows the planned path and the others hold an offset from it. Early work on flocking showed that simple local rules with near neighbours are enough for coherent group motion. This project borrows both ideas: the planner reasons about a leader corridor, and each drone is required to stay in contact only with its immediate neighbours. For curved paths, shortest paths with a bounded turning radius are known as Dubins paths, and such paths are used to join the figure-eight pattern smoothly.

2.5 Fault-Aware Planning for UAVs

Real drones lose GPS, lose radio contact and run out of battery. A plan that ignores these events is not safe to fly. In planning, faults are commonly handled either by monitoring execution and replanning, or by including recovery actions in the domain. This work follows the second idea in a light form: each domain includes detection actions that turn a failed health flag into a failure state and a controller notification, and the point-to-point domain adds explicit controller orders (return, hover, sacrifice). Battery is treated as a numeric resource, so that the planner itself refuses any move the drone cannot afford.

2.6 Position of This Work

The contribution is not a new planning algorithm. It is a family of carefully engineered PDDL models for drone swarm missions, connected to a working simulator, together with an honest record of what the models express, how they scale and where they stop. The models are written to be read by people as well as solved by a planner, so that a supervisor or teammate can check every safety rule directly in the PDDL.

Chapter 3

System Architecture and Planning Pipeline

3.1 Simulator Overview

The simulator has four main parts. The **ground station GUI** (PySide6) shows the map, telemetry, the mission planner panel and fault injection controls. The **plan service** prepares PDDL problems and calls the planner. The **planner** is ENHSP, run as a Java process. The **drones** are independent threads, each with its own state, command inbox and telemetry output. Table 3.1 summarises the roles.

Table 3.1: Components of the simulator and their role in planning

Component	Responsibility
Mission planner panel (GUI)	Choose a named plan, pick source, destination, area or pattern points on the map, set laps and show speed.
Plan service (`services/plan_service.py`)	Select the plan folder, retarget the template to the picked points, expand it to N drones, call ENHSP, check the returned action order and build per-drone routes.
Problem tools (`engine/pddl_problem.py` and per-plan modules)	Rewrite coordinates, distances and energy facts; clone template drones; build neighbour chains and geometry.
ENHSP (`engine/pddl_planner.py`)	Ground and solve the problem; return the plan text, which is parsed into ordered steps.
Thread backend and GCS (`services/thread_backend.py`, `engine/gcs.py`)	Send missions and commands to each drone, receive telemetry and heartbeats, keep formation pace and raise low-battery, GPS-loss and drone-lost events.
Drone thread (`engine/drone_thread.py`)	Integrate motion and battery, monitor health and communication, fly the uploaded route.

3.2 From Map Clicks to Flying Drones

Figure 3.1 shows the planning pipeline. Every plan folder under `plans/` holds one `domain.pddl` and one problem template. The template is written for a small, fixed example (for example one leader and two wings). The pipeline turns this small example into the operator's real mission in eight steps. The step numbers below match the numbers in the figure, and the running example is a V formation of five drones flown around a marked no-fly area.

Step 1. Operator picks points on the map. In the mission planner panel the operator chooses a named plan and the drones to use, then clicks the points the plan needs: a source and a destination, a search area, or the points of a pattern. *Example: the operator chooses "V-Formation", ticks five drones, clicks a source, a destination and draws a no-fly area between them.*

Step 2. The plan folder is chosen. The plan service opens the folder of the chosen plan and reads its domain and its problem template. It decides how to treat the plan from the domain name written inside `domain.pddl`. *Example: the domain name* `v-formation-drone-mission` *tells it that this is a formation plan.*

Step 3. Retarget the template to the map. The template's example coordinates are replaced with the picked points. For every leg the distance is measured again and its `energy-required` is recomputed at that leg's energy-per-metre rate. If a no-fly area was marked and lies in the way, detour locations around it are added, and the direct leg is marked unsafe. *Example: two detour points are added beside the no-fly area, giving the legs source–detour1–detour2–destination.*

Step 4. Expand to *N* drones. The template describes only a few drones. The plan service copies the facts of the template drone, line by line, once for every extra drone, and rebuilds the neighbour facts so that each drone is linked to the right neighbours. *Example: the three-drone V becomes a five-drone V with two ranks per wing, and the wing-neighbour chain becomes leader–rank 1–rank 2 on each side.*

Step 5. ENHSP grounds and searches. The finished problem and the domain are given to ENHSP, which creates every concrete action and searches for an ordered list of actions that reaches the goal while respecting every precondition.

Step 6. Parse and check the plan. The returned plan text is read into ordered steps. The plan service checks that the actions come in the order this kind of plan expects, and extracts the leader's corridor (the list of locations the leader flies through). *Example: the corridor is source, detour1, detour2, destination.*

Step 7. Derive member routes from slots. Every other drone's route is computed from the leader's corridor by shifting it back and sideways by that drone's slot offset. *Example: the rank-2 left drone flies the same corridor, 34.64 m back and 40 m to the left.*

Step 8. Upload one mission per drone and fly. Each route is sent to its drone as one whole mission. The drone threads fly their missions while the ground station monitors telemetry, battery and faults.

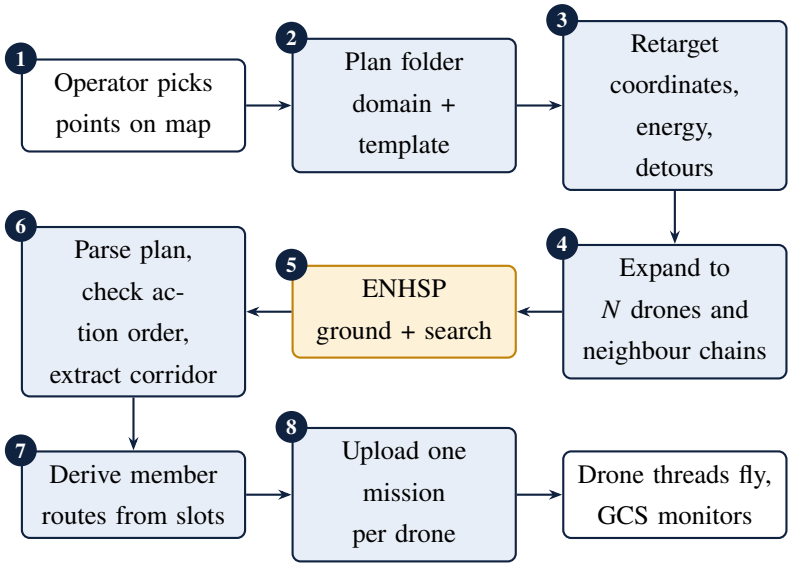


Figure 3.1: The planning pipeline, from the operator's map input to the drones' flight. The numbers are the steps described above.

3.3 Plan Kinds and Folders

The plan service decides how to treat a plan from the *domain name* written inside `domain.pddl`, not from the folder name. This lets a folder be renamed freely and makes the domain file the single source of truth. Table 3.2 lists all ten plans.

Table 3.2: Inventory of the ten mission plans

Folder	Domain name	**Kind**	**Drones**
`travell`	`swarm-drone-mission`	point_to_point	1 to 60
`Search`	`forest-drone-search`	area_coverage	2 to 7
`vformation`	`v-formation-drone-mission`	formation	odd, 3 to 9
`gridformation`	`grid-formation-drone-mission`	grid_formation	4 to 36
`3dgridformation`	`cube-grid-formation-drone-mission`	cube_formation	8 to 64
`parabolaformation`	`parabola-formation-drone-mission`	parabola_formation	exactly 3
`figure8formation`	`figure8-formation-drone-mission`	figure8_formation	2 to capacity
`starformation`	`star-formation-drone-mission`	star_formation	6 to 40
`planets`	`planets-formation-drone-mission`	planets_formation	2 to 8
`Inspect`	`road-inspection-drone-mission`	road_inspection	1

3.4 Run Records

Every planning request writes a time-stamped folder under `plans/<name>/runs/` containing the exact problem that was solved and the full ENHSP output. These records made it possible to trace every unsolvable request back to its input, and they are the source of the statistics in Chapter 8.

Chapter 4

PDDL Modelling Approach

4.1 A Shared Vocabulary

A PDDL domain describes the world with two kinds of building blocks:

- a **predicate** is a yes/no statement, like a light switch that is either on or off. For example, `(gps-ok drone1)` is either true or false;
- a **function** (numeric fluent) is a number that can go up and down, like a fuel gauge. For example, `(battery drone1)` may be 85.

Actions read these switches and gauges in their preconditions and change them in their effects. To make the ten plans easy to read together, I used the same names for the same idea in every domain. A reader who understands one plan can therefore read any other. The domains declare the types `drone`, `location` and `controller` (plus plan-specific types such as `forest-area` or `layer`). Table 4.1 lists the shared names and says in simple words what each one means; `?d` stands for a drone and `?l`, `?a`, `?b` for places.

Table 4.1: Shared predicates and numeric functions used across the domains

Name	Meaning in simple words
Where a drone is and what it is doing (yes/no)	
`(at ?d ?l)`	The drone is at place `?l`.
`(formation-at ?l)`	The whole formation is at place `?l`.
`(landed ?d)`	The drone is on the ground.
`(airborne ?d)`	The drone has taken off.
`(flying ?d)`	The drone is moving on its mission.
`(at-cruise-altitude ?d)`	The drone has climbed to its own working height.
`(connected ?a ?b)`	There is a straight leg from place `?a` to place `?b`.
`(safe-route ?a ?b)`	That leg does not cross a no-fly area.
Health and communication (yes/no)	
`(gps-ok ?d)`	The drone knows where it is: its GPS works.
`(communication-ok ?d)`	The drone can talk to the ground station.
`(neighbor-comm-ok ?d)`	The drone can talk to all its neighbours in the formation.
`(drone-healthy ?d)`	The drone has passed its health check.

Name	Meaning in simple words
`(gps-lost ?d)`	A GPS failure has been detected on this drone.
`(communication-lost ?d)`	A link failure has been detected on this drone.
`(health-failure ?d)`	A health failure has been detected on this drone.
`(controller-notified ?d)`	The ground station has been told about the failure.
Numbers that go up and down	
`(battery ?d)`	Charge left, on a scale where 100 means full.
`(max-battery ?d)`	The charge of a full battery.
`(minimum-return-battery ?d)`	Charge that must always be kept to get home safely.
`(distance ?a ?b)`	Length of a leg, in metres.
`(energy-required ?a ?b)`	Charge used to fly that leg.
`(climb-energy ?d)`	Charge used to climb to working height.
`(cruise-leg-energy ?d)`	Charge a follower uses for one leg of the formation.
`(health ?d)`	Health score of the drone (full health is 100).
`(minimum-health ?d)`	Lowest health score allowed for flying.
`(altitude ?d)`	Current height of the drone.
`(hover-altitude ?d)`	Height right after take-off.
`(target-altitude ?d)`	Working (cruise) height of this drone.
`(slot-along-offset ?d)`	How far behind the leader the drone flies, in metres.
`(slot-cross-offset ?d)`	How far to the left or right of the leader it flies.

4.2 The Common Mission Life Cycle

Every formation plan follows the same life cycle (Figure 4.1). A drone first passes a `pre-flight-check`, which needs it to be landed at the source with GPS, radio and health in order and a battery above its return reserve. It then takes off to hover altitude and climbs to its own cruise altitude, paying `climb-energy`. Once airborne, it takes its slot in the formation. The establish action (for example `establish-v-formation`) fires only when every member is in its slot, at altitude and in contact with its neighbours. The leader then flies the corridor one leg at a time with `formation-cruise`; after each leg the formation is marked "open" and a close-up action (`close-up-wings`, `close-up-grid`, `hold-parabola-shape`, `follow-leader`) must bring every member back into its slot before the next leg. The mission completes when the closed formation is at the destination.

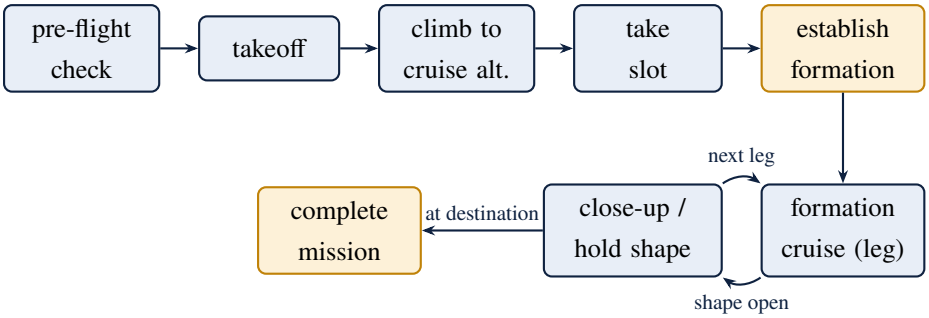


Figure 4.1: The life cycle shared by the formation plans. Each box is one PDDL action (or one action per drone).

The take-off step itself is short but illustrates the style used throughout:

```
(:action takeoff
    :parameters (?d - drone ?l - location)
    :precondition (and (at ?d ?l) (source ?l) (landed ?d)
                       (preflight-done ?d)
                       (gps-ok ?d) (communication-ok ?d) (drone-healthy ?d))
    :effect (and (not (landed ?d)) (airborne ?d) (flying ?d)
                 (assign (altitude ?d) (hover-altitude ?d))))
```

In words: a drone may take off only if it is on the ground at the source, has passed its pre-flight check, and its GPS, radio and health are all fine. After take-off it is no longer landed, it is airborne and flying, and its height is set to the hover height.

4.3 Design Patterns for Tractable Swarm Models

Swarm problems grow quickly with the number of drones: every extra drone adds more actions for the planner to try. Four simple ideas kept the models small enough for ENHSP to solve within seconds. Each is explained below as the problem, the idea, and why it helps.

1. Plan the leader, let the others follow. *The problem:* if the planner had to choose a route for every drone separately, its work would grow very fast with each extra drone. *The idea:* think of a mother duck and her ducklings. Only the mother decides where to go; each duckling simply keeps its place behind her. In the same way ENHSP plans only the leader's path. After every leg, one single action moves all followers back into their places at once. The followers' real map routes are worked out afterwards, outside the planner, by shifting the leader's route back and sideways by each drone's slot. *Why it helps:* the planning work grows only a little with each extra drone.

2. One "can talk to my neighbours" flag per drone. *The problem:* checking every *pair* of drones is a lot of checks. A cube of 64 drones has hundreds of neighbour pairs, and checking them all in every action would slow the planner down. *The idea:* the problem file still lists who is a neighbour of whom (`wing-neighbor`, `grid-neighbor`, `cube-neighbor`, `train-neighbor`, `orbit-neighbor`), so that a person or the run time can read it. But the actions check only one yes/no flag per drone,

`neighbor-comm-ok`, which means "this drone can talk to all of its neighbours". *Why it helps:* the number of checks grows with the number of drones, not with the number of pairs.

3. On/off steps instead of comparing two counters. *The problem:* the cube takes off layer by layer, bottom first. My first version used a counter "layers cleared so far" and let a drone take off only when its own layer number was not larger than that counter. ENHSP is poor at judging progress when two changing numbers are compared with each other, so it searched almost blindly and a 25-drone cube did not finish in 90 s. *The idea:* think of a traffic light for each floor of a building. Each layer became an object with a "launched" light, and a layer's light turns green only when the layer below has fully climbed. *Why it helps:* the planner can now see clear progress, and the same problem was solved in about one second.

4. Write one fact per line in templates. *The problem:* the program that grows a template to N drones copies it line by line. If one fact is split over three lines, only part of it is copied and the new drone gets a broken fact. *The idea:* always write each per-drone fact on a single line, for example `(= (hover-altitude drone-member) 2)`. *Why it helps:* copying a line always copies a whole fact. Breaking this rule caused the grid formation defect described in Section 8.7.

Chapter 5

The Mission Plans

This chapter explains each of the ten plans in the same way:

- **What the mission does**, in plain words, and an *at a glance* box with its map input, number of drones and neighbours;
- **Communication**: a diagram of which drones talk to which, followed by the PDDL that expresses it;
- **Important PDDL**: only the actions that make this plan different from the others. The parts every plan shares (pre-flight check, take-off, climb, fault detection) are explained once, in Chapter 4 and Chapter 7;
- **Action order**: the order in which ENHSP applies the actions;
- **Output**: what the plan looks like in the simulator.

Figure 5.1 explains the symbols used in all communication diagrams of this chapter.

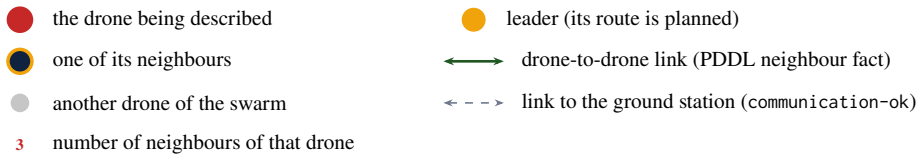


Figure 5.1: Symbols used in the communication diagrams. Every drone in every plan also has a ground-station link, even where it is not drawn.

5.1 Point-to-Point Travel

What the mission does. The whole selected swarm flies one shared route from a picked source to a picked destination. If the operator marks a no-fly area across the direct line, the route goes around it through detour waypoints. The template has a single drone, `drone1`, which is copied once for every selected drone, so every drone gets its own checks and its own legs in the plan. This is the base domain of the project, and it also contains the most complete fault handling (Chapter 7).

At a glance: point-to-point travel	
Map input	source, destination, optional no-fly area
Drones	1 to 60, any number in between
Neighbours per drone	none – each drone talks only to the ground station
ENHSP plans	every drone's legs
Special feature	battery check before every leg; full ground-controller fault responses

Communication. The drones fly the same route independently and do not depend on each other, so no drone-to-drone links are needed. Each drone has exactly one link, to the ground station (Figure 5.2).

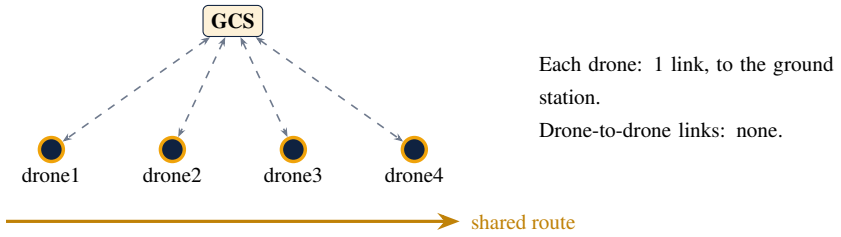


Figure 5.2: Communication in point-to-point travel: every drone talks only to the ground station.

In PDDL the ground link is the flag `communication-ok`. The problem starts with it set for every drone, and the moving actions refuse to move a drone without it:

```
;; problem: the template drone (copied for every drone) starts with a working link
(gps-ok drone1)
(communication-ok drone1)

;; domain: a leg is flown only while the link (and GPS, health, battery) is fine
(:action travel
    :parameters (?d - drone ?from - location ?to - location)
    :precondition (and (at ?d ?from) (route-planned ?d) (safe-route ?from ?to)
                       (gps-ok ?d) (communication-ok ?d) (drone-healthy ?d)
                       (> (battery ?d) (energy-required ?from ?to)))
    :effect (and (at ?d ?to) (not (at ?d ?from)) (flying ?d)
                 (decrease (battery ?d) (energy-required ?from ?to))
                 (not (route-planned ?d))))
```

Important PDDL. Before each move a drone performs `check-distance` and `check-battery`; if the battery cannot cover the leg, `notify-insufficient-battery` records the fact instead. A route is planned only when every safety check passes:

```
(:action plan-safe-route
    :parameters (?d - drone ?from - location ?to - location)
    :precondition (and (at ?d ?from) (connected ?from ?to) (safe-route ?from ?to)
                       (battery-checked ?d)
                       (gps-ok ?d) (communication-ok ?d) (drone-healthy ?d))
    :effect (route-planned ?d))
```

The `travel` action above clears `route-planned`, so the checks must be repeated for every leg. A direct leg that crosses the no-fly area is marked `restricted-route` and has no `safe-route` fact, so the planner has to use the detour waypoints.

Action order. For each drone:

```
check-distance → check-battery → plan-safe-route → travel → reach-destination
```

With detours, the middle three actions repeat once per leg.

Reading the ENHSP output. Below is the complete output of one logged request with three drones and no no-fly area (the lines in the middle of the search are shortened).

ENHSP output — point-to-point travel, three drones

```
Domain parsed
Problem parsed
Grounding..
Grounding Time: 26
|F|:24
|X|:3
|A|:36
h(I):27.0
 g(n)= 1.0 h(n)=25.0
 g(n)= 2.0 h(n)=23.0
 ...
 g(n)= 14.0 h(n)=1.0
Problem Solved

Found Plan:
0.0: (check-distance drone3 source destination)
1.0: (check-distance drone2 source waypoint1)
2.0: (check-distance drone1 source waypoint1)
3.0: (check-battery drone1 source waypoint1)
4.0: (check-battery drone2 source waypoint1)
5.0: (check-battery drone3 source destination)
6.0: (plan-safe-route drone3 source destination)
7.0: (plan-safe-route drone2 source destination)
8.0: (plan-safe-route drone1 source destination)
9.0: (travel drone2 source destination)
10.0: (travel drone1 source destination)
11.0: (travel drone3 source destination)
12.0: (reach-destination drone2 destination)
13.0: (reach-destination drone1 destination)
14.0: (reach-destination drone3 destination)

Plan-Length:15
Planning Time (msec): 34
Expanded Nodes:16
States Evaluated:43
```

The output has three parts.

1. **Preparation.** ENHSP reads the two files and then *grounds* them: it replaces every `?d`, `?from` and `?to` by real drones and places, producing every concrete action (26 ms here). `|F|:24` means there are 24 yes/no facts, `|X|:3` means there are 3 numbers (the battery of each drone) and `|A|:36` means there are 36 concrete actions to choose from.

2. **Search.** `h(I):27.0` is ENHSP's estimate of how far the start is from the goal. Each line `g(n)= … h(n)= …` shows the number of actions chosen so far (g) and the estimated number still needed (h). Here h falls steadily from 25 to 1, which means the search went straight to the goal without getting lost.

3. **The plan.** Each line has the form `step: (action arguments)`. The number before the colon is the *position* of the action in the plan (0, 1, 2, …), not a time in seconds. Reading only the lines of `drone1` gives its own story: it checks the distance, checks its battery, plans a safe route, travels from the source to the destination and reaches the destination. ENHSP mixes the three drones together because their order does not matter; the plan service separates them again by drone. `Plan-Length:15` is five actions for each of the three drones, found in 34 ms after evaluating only 43 states.

One detail is worth noticing: `drone1` and `drone2` checked their battery against the leg to `waypoint1` but then flew straight to the destination. This is allowed because the check flags belong to the drone, not to one leg. It is still safe, because the `travel` action itself checks again that the battery covers the leg that is actually flown.



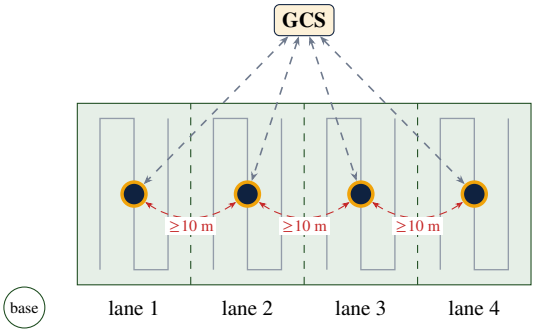
Figure 5.3: Point-to-point travel in the simulator: the shared route from source to destination, flown by all selected drones.

5.2 Forest Area Search

What the mission does. The operator clicks a base point and the corners of a forest area. The area is split into equal lanes, one per drone, and each lane is covered by a back-and-forth ("lawnmower") path. Every drone takes off from the common base, flies to its own lane, covers it, and returns to the base. Unlike travel, each drone has its own route. At least two drones are needed.

At a glance: forest area search	
Map input	base point, corners of the forest area
Drones	2 to 7 (one lane per drone)
Neighbours per drone	none – ground station link, plus a separation check against every other drone
ENHSP plans	one lane per drone, out and back
Special feature	pairwise minimum separation of 10 m; low-battery return

Communication. Each drone talks only to the ground station. The drones do not exchange messages, but the planner makes sure that no two drones come closer than a minimum separation (Figure 5.4).



Each drone: 1 link, to the ground station.

Red: the planner checks that every pair of drones keeps at least the minimum separation.

Figure 5.4: Communication in forest area search: ground links only, plus a separation check between drones.

In PDDL the separation is a number for every pair of drones, and an action confirms that the pair is far enough apart. The covering action needs the drone's ground link:

```
;; problem: minimum distance between the two template drones, in metres
(= (minimum-drone-separation drone1 drone2) 10)
(= (minimum-drone-separation drone2 drone1) 10)

;; domain: a pair of drones is marked safe only if they are far enough apart
(:action check-drone-separation
    :parameters (?d1 - drone ?d2 - drone ?l1 - location ?l2 - location)
    :precondition (and (at ?d1 ?l1) (at ?d2 ?l2) (not (= ?d1 ?d2))
                       (>= (distance ?l1 ?l2) (minimum-drone-separation ?d1 ?d2)))
    :effect (and (safe-separation ?d1 ?d2) (separation-ok ?d1) (separation-ok ?d2)))
```

Important PDDL. The type `forest-area` represents a lane, and `(assigned ?d ?a)` gives each drone its own lane. A lane is marked covered only by the drone assigned to it, while its GPS, link and health are fine:

```
(:action cover-area
    :parameters (?d - drone ?a - forest-area ?l - location)
    :precondition (and (at ?d ?l) (forest ?a) (area-location ?a ?l) (assigned ?d ?a)
                       (searching ?d) (not (area-covered ?a))
                       (gps-ok ?d) (communication-ok ?d) (drone-healthy ?d))
    :effect (and (area-covered ?a) (search-completed ?d) (not (searching ?d))))
```

An `emergency-low-battery-return` action marks a drone as returning whenever its battery falls to the return reserve.

Action order. For each drone:

```
check-battery → start-search → move-search (repeated along the lane) → cover-area →
complete-forest-search → return-to-base → complete-drone-mission
```

Simulator output for this plan

Screenshot to be added: `figures/screenshots/search.png`

Figure 5.5: Forest area search in the simulator: the marked area split into one lane per drone.

5.3 V Formation

What the mission does. The operator picks points as for travel. A leader (the apex) flies the corridor and the other drones form two wings behind it, like a flock of birds. Each rank sits 17.32 m back and 20 m to the side of the one before it, so the wing angle is 60° (Figure 5.9a). The number of drones must be odd: one leader plus the same number on each wing. Extra drones add further ranks. The leader takes off first and the wings follow after a short delay.

At a glance: V formation	
Map input	source, destination, optional no-fly area
Drones	odd numbers, 3 to 9
Neighbours per drone	1 or 2 – the drone in front and the drone behind on its wing
ENHSP plans	the leader's corridor
Special feature	timed wing launch; wings close up after every leg

Communication. Each wing is a chain. A drone talks to the drone in front of it and the drone behind it on the same wing. The leader talks to the first drone of each wing (Figure 5.6).

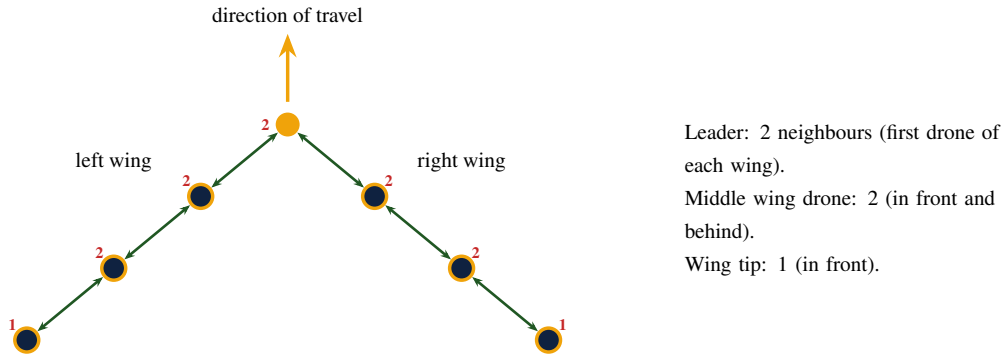


Figure 5.6: Communication in the V formation: each wing is a chain of neighbours starting at the leader.

In PDDL the chain is written as `wing-neighbor` facts in both directions, and every drone has the `neighbor-comm-ok` flag. The formation can only be established when every wing drone has that flag:

```
;; problem: leader <-> rank 1 on each side (the expansion code adds rank 1 <-> rank 2, ...)
(wing-neighbor drone-lead drone-left)
(wing-neighbor drone-left drone-lead)
(wing-neighbor drone-lead drone-right)
(wing-neighbor drone-right drone-lead)
(neighbor-comm-ok drone-lead)
(neighbor-comm-ok drone-left)
(neighbor-comm-ok drone-right)

;; domain: the V is formed only if every wing drone is in place AND in contact
(:action establish-v-formation
    :parameters (?lead - drone ?l - location)
    :precondition (and (formation-leader ?lead) (slot-apex ?lead) (in-formation ?lead)
                       (at ?lead ?l) (at-cruise-altitude ?lead) (neighbor-comm-ok ?lead)
                       (forall (?w - drone)
                           (or (not (wing-drone ?w))
                               (and (in-formation ?w) (at ?w ?l)
                                    (at-cruise-altitude ?w) (neighbor-comm-ok ?w)))))
    :effect (and (v-formation-established) (formation-at ?l) (wings-closed)))
```

Important PDDL. The timed launch uses a counter. While the airborne leader waits, `hold-position` adds one second until the wing launch delay is reached; only then may the wing drones take off:

```
(:action hold-position
    :parameters (?lead - drone)
    :precondition (and (formation-leader ?lead) (airborne ?lead)
                       (< (seconds-since-leader-airborne) (wing-launch-delay))
                       (gps-ok ?lead) (communication-ok ?lead) (drone-healthy ?lead))
    :effect (increase (seconds-since-leader-airborne) 1))
```

After each leg, `close-up-wings` moves every wing drone to the leader's new location in one step, using a quantified conditional effect, and charges each of them `cruise-leg-energy`.

Action order. ENHSP applies the actions in this order:

```
pre-flight-check, takeoff, climb (leader) → hold-position (until the delay) → pre-flight-check,
takeoff, climb (each wing drone) → take-formation-slot (each drone) → establish-v-formation →
[formation-cruise → close-up-wings] for every leg → complete-formation-mission
```

Simulator output for this plan

Screenshot to be added: `figures/screenshots/vformation.png`

Figure 5.7: V formation in the simulator: the leader's corridor and the wing routes derived from it.

5.4 Two-Dimensional Grid Formation

What the mission does. The drones fly as a flat grid, 10 m apart in every direction (Figure 5.9b). A square number of drones gives a square grid; otherwise the most square rectangle that fits the number exactly is used (for example 12 drones become 3×4). At least four drones are needed. The leader flies in the front row and the rest of the grid moves with it.

At a glance: two-dimensional grid	
Map input	source, destination, optional no-fly area
Drones	4 to 36
Neighbours per drone	2 to 4 – front, back, left and right
ENHSP plans	the leader's corridor
Special feature	grid shape chosen from the drone count; close-up after every leg

Communication. Each drone talks only to the drones directly in front, behind, to the left and to the right. A corner drone therefore has 2 neighbours, an edge drone 3 and an inner drone 4 (Figure 5.8).

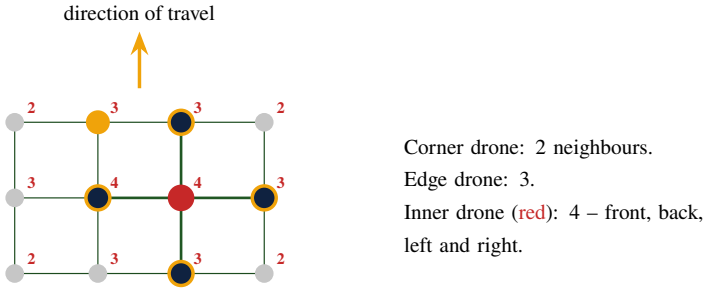


Figure 5.8: Communication in the grid formation: each drone talks to at most four drones next to it.

In PDDL this is the `grid-neighbor` relation. The close-up after every leg checks that every member still has its `neighbor-comm-ok` flag:

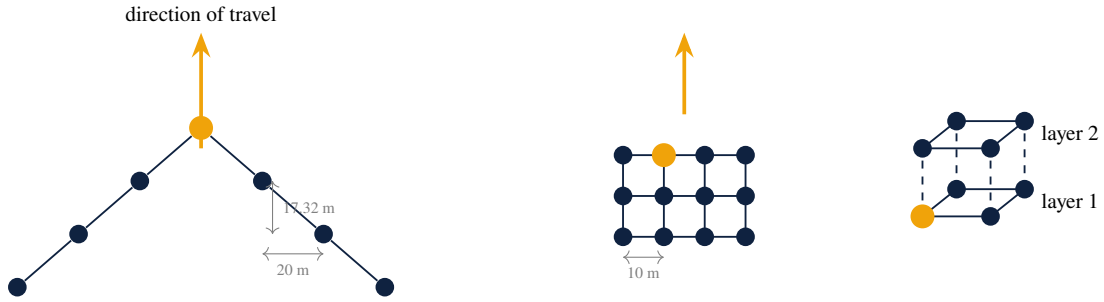
```
;; problem: one neighbour pair of the template (rebuilt for the full grid on expansion)
(grid-neighbor drone-lead drone-member)
(grid-neighbor drone-member drone-lead)
(neighbor-comm-ok drone-lead)
(neighbor-comm-ok drone-member)

;; domain: after a leg, the grid is closed again only if every member is still fine
(:action close-up-grid
    :parameters (?lead - drone ?to - location)
    :precondition (and (grid-formation-established) (not (grid-closed))
                       (formation-leader ?lead) (formation-at ?to) (at ?lead ?to)
                       (forall (?m - drone)
                           (or (not (grid-member ?m))
                               (and (in-formation ?m) (at-cruise-altitude ?m)
                                    (gps-ok ?m) (communication-ok ?m) (drone-healthy ?m)
                                    (neighbor-comm-ok ?m)
                                    (> (battery ?m) (cruise-leg-energy ?m))))))
    :effect (and (forall (?m - drone) (when (grid-member ?m) (at ?m ?to)))
                 (forall (?m - drone)
                     (when (grid-member ?m) (decrease (battery ?m) (cruise-leg-energy ?m))))
                 (grid-closed)))
```

Important PDDL. The grid differs from the V mainly in its slot offsets and neighbour relation: every member has its own `slot-along-offset` and `slot-cross-offset` (multiples of 10 m), and the close-up action above is the only action that moves members. Because the planner moves all members in one action, the plan length grows by only about four actions per extra drone.

Action order. ENHSP applies the actions in this order:

```
pre-flight-check, takeoff, climb (leader) → hold-position → pre-flight-check, takeoff, climb
(each member) → take-grid-slot (each drone) → establish-grid-formation → [formation-cruise →
close-up-grid] for every leg → complete-grid-mission
```



(a) V formation (b) 3×4 grid (c) $2 \times 2 \times 2$ cube

Figure 5.9: Rigid formations. The amber drone is the leader whose corridor ENHSP plans; lines show the neighbour links each drone must keep. In the cube, vertical layers are separated by height.

Simulator output for this plan

Screenshot to be added: `figures/screenshots/grid.png`

Figure 5.10: Grid formation in the simulator: one route per grid cell, parallel to the leader's corridor.

5.5 Three-Dimensional Cube Formation

What the mission does. The grid extended upwards: a solid block of drones flies one corridor, 10 m apart horizontally and vertically (Figure 5.9c). A square footprint is chosen and as many layers as needed are stacked, allowing a partly filled top layer (for example 9 drones become $3 \times 2 \times 2$ with 9 of 12 cells filled; 25 become $3 \times 3 \times 3$). At least eight drones are needed. To avoid collisions while climbing, the block takes off layer by layer: the bottom layer first, and each next layer only when the one below has reached its height.

At a glance: cube formation	
Map input	source, destination, optional no-fly area
Drones	8 to 64
Neighbours per drone	up to 24 – the surrounding $3 \times 3 \times 3$ block, except directly above and below
ENHSP plans	the leader's corridor and the layer-by-layer launch
Special feature	each layer at its own height; layer gates for the climb

Communication. A drone talks to the drones around it in its own layer (up to 8) and in the layers just above and below (up to 8 each). The drones directly above and below are not counted as neighbours, because the vertical spacing alone keeps them apart. That gives up to 24 neighbours (Figure 5.11).

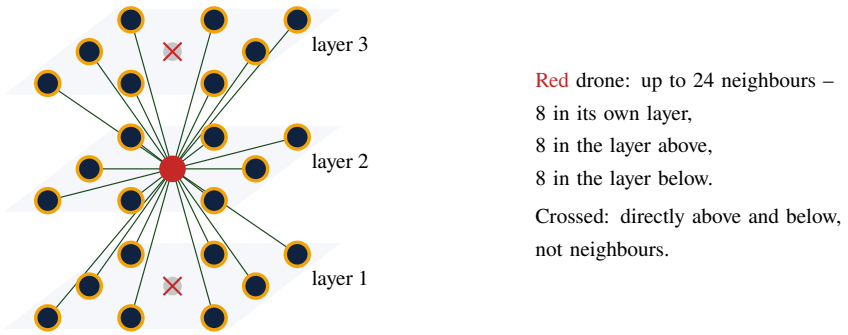


Figure 5.11: Communication in the cube formation: up to 24 neighbours in the surrounding block.

In PDDL the neighbours are `cube-neighbor` facts, and, as in the grid, the formation is established only when every member has `neighbor-comm-ok`. The cube adds one more condition: every layer must already have been launched:

```
;; problem: one neighbour pair of the template (all 24-neighbour pairs added on expansion)
(cube-neighbor drone-lead drone-member)
(cube-neighbor drone-member drone-lead)
(neighbor-comm-ok drone-lead)
(neighbor-comm-ok drone-member)

;; domain: the block is formed only when all layers are up and every member is in contact
(:action establish-cube-formation
    :parameters (?lead - drone ?l - location)
    :precondition (and (formation-leader ?lead) (in-formation ?lead) (at ?lead ?l)
                       (at-cruise-altitude ?lead) (neighbor-comm-ok ?lead)
                       (forall (?ly - layer) (layer-launched ?ly))
                       (forall (?m - drone)
                           (or (not (cube-member ?m))
                               (and (in-formation ?m) (at ?m ?l)
                                    (at-cruise-altitude ?m) (neighbor-comm-ok ?m)))))
    :effect (and (cube-formation-established) (formation-at ?l) (cube-closed)))
```

Important PDDL. Layers are separated by height, so each drone has its own `target-altitude` (base plus 10 m per layer), and the run-time route adds a per-drone altitude offset. The swarm climbs layer by layer using the "traffic light" pattern of Chapter 4. A layer is opened only when every drone of the layer below has reached its height, and opening it clears all of its drones for take-off:

```
(:action open-next-layer
    :parameters (?lo - layer ?hi - layer)
    :precondition (and (layer-below ?lo ?hi) (layer-launched ?lo)
                       (not (layer-launched ?hi))
                       (forall (?d - drone)
                           (or (not (in-layer ?d ?lo)) (at-cruise-altitude ?d))))
    :effect (and (layer-launched ?hi)
                 (forall (?d - drone)
                     (when (in-layer ?d ?hi) (cleared-to-launch ?d)))))
```

Collision avoidance is built into the structure: fixed spacing, layer-by-layer ascent and a `cube-closed` interlock that must hold before each leg.

Action order. ENHSP applies the actions in this order:

```
launch layer 1 (pre-flight-check, takeoff, climb for each drone) → open-next-layer → launch layer
2 → … → take-cube-slot (each drone) → establish-cube-formation → [formation-cruise →
close-up-cube] for every leg → complete-cube-mission
```

Simulator output for this plan

Screenshot to be added: `figures/screenshots/cube.png`

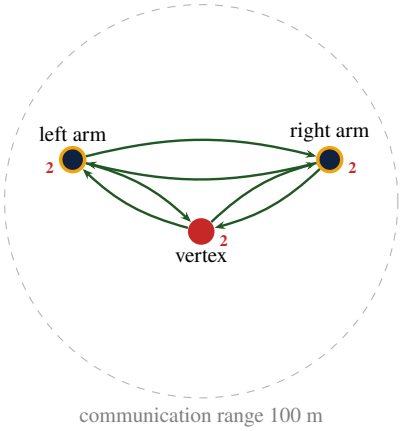
Figure 5.12: Cube formation in the simulator: the layers of the block flying one corridor at different heights.

5.6 Parabola Formation

What the mission does. Three drones fly side by side on one line across the route, 10 m apart. The middle drone (the vertex) cruises at 50 m and the two arm drones at 60 m, so seen from behind they form an upward-opening parabola (Figure 5.14). Extra selected drones stay on the ground. This plan has the most detailed communication model: the drones must set up radio links with each other, within range, before they may take their places.

At a glance: parabola formation	
Map input	source, destination, optional no-fly area
Drones	exactly 3
Neighbours per drone	2 – every drone talks to both others (6 directed links)
ENHSP plans	the vertex corridor and every link set-up
Special feature	range-checked links; the shape itself is checked in PDDL

Communication. All three pairs keep a direct link, in both directions, so there are six directed links (Figure 5.13). A link can only be set up when the two drones are within the communication range of 100 m.



Every drone: 2 neighbours.
6 directed links, one in each direction for each pair.
Each link is set up only within range.

Figure 5.13: Communication in the parabola formation: a full mesh of six directed, range-checked links.

In PDDL, every directed link is a `parabola-neighbor` fact, and a link becomes `link-up` only through `establish-comm-link`. A drone is ready only when it has a link to every one of its neighbours:

```
;; problem: all six directed neighbour pairs
(parabola-neighbor drone-vertex drone-left)   (parabola-neighbor drone-left drone-vertex)
(parabola-neighbor drone-vertex drone-right)  (parabola-neighbor drone-right drone-vertex)
(parabola-neighbor drone-left drone-right)    (parabola-neighbor drone-right drone-left)

;; domain: a link is set up only between airborne drones that are within range
(:action establish-comm-link
    :parameters (?d1 - drone ?d2 - drone)
    :precondition (and (parabola-neighbor ?d1 ?d2) (not (link-up ?d1 ?d2))
                       (airborne ?d1) (airborne ?d2)
                       (at-cruise-altitude ?d1) (at-cruise-altitude ?d2)
                       (communication-ok ?d1) (communication-ok ?d2)
                       (neighbor-comm-ok ?d1) (neighbor-comm-ok ?d2)
                       (<= (link-distance ?d1 ?d2) (comm-range)))
    :effect (link-up ?d1 ?d2))

;; domain: a drone is ready when every one of its links is up
(:action confirm-swarm-comm
    :parameters (?d - drone)
    :precondition (and (airborne ?d) (not (swarm-comm-ready ?d))
                       (communication-ok ?d) (neighbor-comm-ok ?d)
                       (forall (?n - drone)
                           (or (not (parabola-neighbor ?d ?n)) (link-up ?d ?n))))
    :effect (swarm-comm-ready ?d))
```

Important PDDL. The formation action checks the shape itself: each arm's target altitude minus the vertex's must equal `arm-rise` (10 m), so a flat line, an upside-down U or lopsided arms give no plan, and every vertex–arm gap (14.14 m) must be at least `min-separation` (10 m).

```
(:action establish-parabola-formation
  :parameters (?vtx - drone ?l - location)
  :precondition (and (formation-leader ?vtx) (slot-vertex ?vtx) (in-formation ?vtx)
     (at ?vtx ?l) (at-cruise-altitude ?vtx) (swarm-comm-ready ?vtx)
     (forall (?a - drone)
       (or (not (arm-drone ?a))
           (and (in-formation ?a) (at ?a ?l) (at-cruise-altitude ?a)
                (= (- (target-altitude ?a) (target-altitude ?vtx)) (arm-rise))
                (>= (link-distance ?vtx ?a) (min-separation))
                (swarm-comm-ready ?a))))))
```

```
:effect (and (parabola-established) (formation-at ?l) (shape-held)))
```

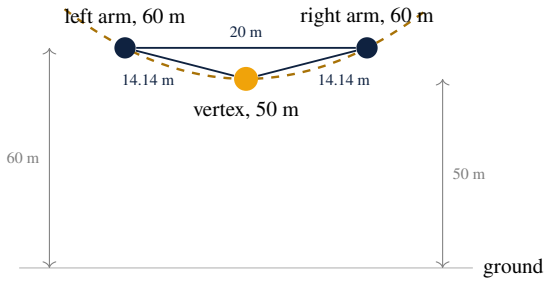


Figure 5.14: The parabola formation seen from behind. All three pairs keep a direct link (full mesh), and every link is within the 100 m communication range.

At run time the vertex corridor is split into legs of at most 200 m, and the arms are derived parallel at ±10 m. All three drones fly one shared timetable, so they arrive at each waypoint together even though the vertex has 10 m less to climb.

Action order. ENHSP applies the actions in this order:

```
pre-flight-check, takeoff, climb (each drone) → establish-comm-link (six times) and
confirm-swarm-comm (three times) → take-parabola-slot (each drone) → establish-parabola-formation
→ [formation-cruise → hold-parabola-shape] for every leg → complete-parabola-mission
```

A complete solved plan is shown in Section 8.6.

Simulator output for this plan

Screenshot to be added: `figures/screenshots/parabola.png`

Figure 5.15: Parabola formation in the simulator: the vertex route and the two arm routes beside it.

5.7 Figure-Eight Train Formation

What the mission does. The swarm flies a figure-eight in the air as a *train*: one drone behind another along the curve, 10 m apart. The operator makes four clicks: a base, then the left end, the crossing point and the right end of the eight. Each lobe is half a lemniscate of Gerono from the crossing to its clicked end, so the lobes cross at right angles and may differ in size (Figure 5.17). The drones fly from the base to the crossing along a Dubins path (turn radius at least 25 m), fly the requested number of laps, and return to their own landing pads.

At a glance: figure-eight train

Map input	base, left end, crossing, right end; number of laps
Drones	2 up to the capacity of the drawn eight (27 on a typical eight)
Neighbours per drone	1 or 2 – the drone in front and the drone behind
ENHSP plans	the leader's lap and the lap count
Special feature	capacity rule checked in PDDL; laps counted, not unrolled

Communication. Like the carriages of a train, each drone talks only to the drone directly in front of it and the drone directly behind it. The leader and the last drone have one neighbour each (Figure 5.16).

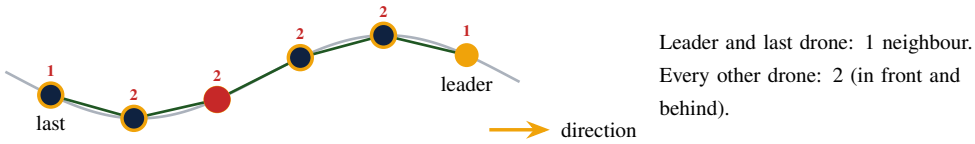


Figure 5.16: Communication in the figure-eight train: each drone talks to the drone in front and the drone behind.

In PDDL the chain is the `train-neighbor` relation. Forming the train needs every follower to have `neighbor-comm-ok`, and also checks the capacity rule:

```
;; problem: leader <-> first follower (the chain is extended for every extra drone)
(train-neighbor drone-lead drone-f1)
(train-neighbor drone-f1 drone-lead)
(neighbor-comm-ok drone-lead)
(neighbor-comm-ok drone-f1)

;; domain: the train is formed only if it fits the eight and everyone is in contact
(:action establish-figure8-formation
    :parameters (?lead - drone ?l - location)
    :precondition (and (train-leader ?lead) (in-formation ?lead) (at ?lead ?l)
                       (at-cruise-altitude ?lead) (neighbor-comm-ok ?lead)
                       (>= (crossing-gap) (crossing-clearance))
                       (forall (?f - drone)
                           (or (not (train-follower ?f))
                               (and (in-formation ?f) (at ?f ?l)
                                    (at-cruise-altitude ?f) (neighbor-comm-ok ?f)))))
    :effect (and (figure8-established) (formation-at ?l) (train-closed)))
```

Important PDDL. The leader's lap is planned through nine key waypoints with `formation-cruise`, and `follow-leader` moves the followers after every leg. Laps are counted with a number, so that 20 laps do not make the plan 20 times longer:

```
(:action start-next-lap
  :parameters (?lead - drone ?dest - location ?start - location)
  :precondition (and (lap-end ?dest) (lap-start ?start) (figure8-established)
                     (train-closed) (formation-at ?dest) (train-leader ?lead)
                     (at ?lead ?dest) (< (+ (laps-completed) 1) (laps-required)))
  :effect (and (not (formation-at ?dest)) (formation-at ?start)
               (not (at ?lead ?dest)) (at ?lead ?start)
               (increase (laps-completed) 1)))
```

Capacity. The only place the train can meet itself is the crossing. The train length $(N-1) \times 10\,\text{m}$ plus 14.1 m of crossing clearance must therefore fit inside the smaller lobe. This rule is checked in the

GUI before planning, again in the plan worker, and inside PDDL itself (the `crossing-gap` condition above), so ENHSP refuses an over-full eight.

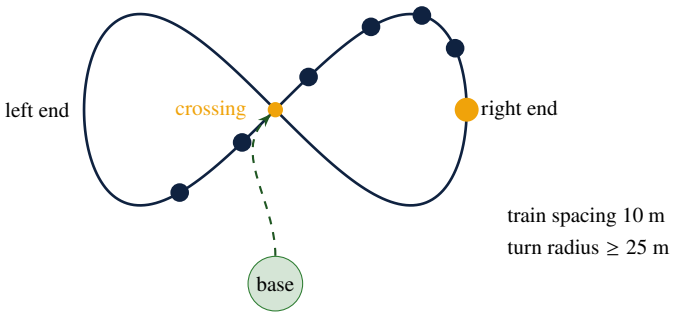


Figure 5.17: The figure-eight train. The leader (amber) and followers fly the same path one spacing apart; the base leg joins the eight tangentially at the crossing.

Action order. ENHSP applies the actions in this order:

```
pre-flight-check, takeoff, climb (each drone) → take-train-slot (each drone) →
establish-figure8-formation → fly-to-pattern → [formation-cruise → follow-leader] for the nine
legs of a lap → start-next-lap (repeat the lap) → finish-laps → return-to-base →
complete-figure8-mission
```

Simulator output for this plan

Screenshot to be added: `figures/screenshots/figure8.png`

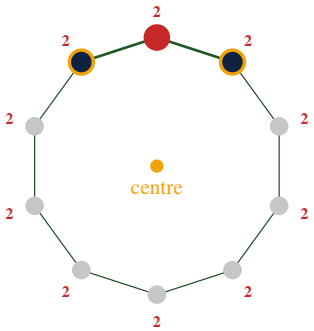
Figure 5.18: Figure-eight train in the simulator: the drones flying the eight one behind another.

5.8 Star Drone Show

What the mission does. A stationary light show around one picked point. The drones take off from pads in rings, spread into a circle, and then, for each lap, form a star, return to the circle, spin the circle once, form the star again, spin the star and return to the circle, before landing on their own pads (Figure 5.22a). Six or seven drones make a 3-pointed star, eight or nine a 4-pointed star, and ten or more a 5-pointed star.

At a glance: star drone show	
Map input	one centre point; number of laps and show speed
Drones	6 to 40
Neighbours per drone	2 – the drones on either side round the circle
ENHSP plans	the show phases, in order
Special feature	drones keep their order round the centre, so shape changes never cross

Communication. The drones stand in a ring. Each drone talks to the two drones on either side of it round the circle, so every drone has exactly two neighbours (Figure 5.19).



Every drone: 2 neighbours, one on each side round the circle.

Figure 5.19: Communication in the star show: a ring in which every drone talks to its two neighbours.

In PDDL this is the `ring-neighbor` relation. The show can only start when every drone has climbed and has `neighbor-comm-ok`:

```
;; problem: one ring pair of the template (the full ring is built on expansion)
(ring-neighbor drone-lead drone-m1)
(ring-neighbor drone-m1 drone-lead)
(neighbor-comm-ok drone-lead)
(neighbor-comm-ok drone-m1)

;; domain: the circle is formed only when every member is up and in contact
(:action form-circle
    :parameters (?lead - drone)
    :precondition (and (formation-leader ?lead) (at-cruise-altitude ?lead)
                       (neighbor-comm-ok ?lead) (not (show-started))
                       (forall (?m - drone)
                           (or (not (star-member ?m))
                               (and (at-cruise-altitude ?m) (neighbor-comm-ok ?m)))))
    :effect (and (show-started) (circle-formed)))
```

Important PDDL. Each phase of the show is one action that switches the current shape, for example `form-star`, which turns the circle into a star at the start of a lap:

```
(:action form-star
    :parameters (?lead - drone)
    :precondition (and (formation-leader ?lead) (circle-formed) (not (lap-started))
                       (< (laps-completed) (laps-required))
                       (gps-ok ?lead) (communication-ok ?lead))
    :effect (and (not (circle-formed)) (star-formed) (lap-started)))
```

The plan worker checks that ENHSP returns exactly the expected sequence. Every drone keeps the same order round the centre in the circle and in the star, so shape changes never cross paths; the show is sized so that all drones are at least 10 m apart in each shape and at least 9 m apart during transitions.

Action order. ENHSP applies the actions in this order:

```
pre-flight-check, takeoff, climb (each drone) → form-circle → [form-star → form-circle-in-lap
→ spin-circle → form-star-again → spin-star → close-lap] for every lap → land-show
```

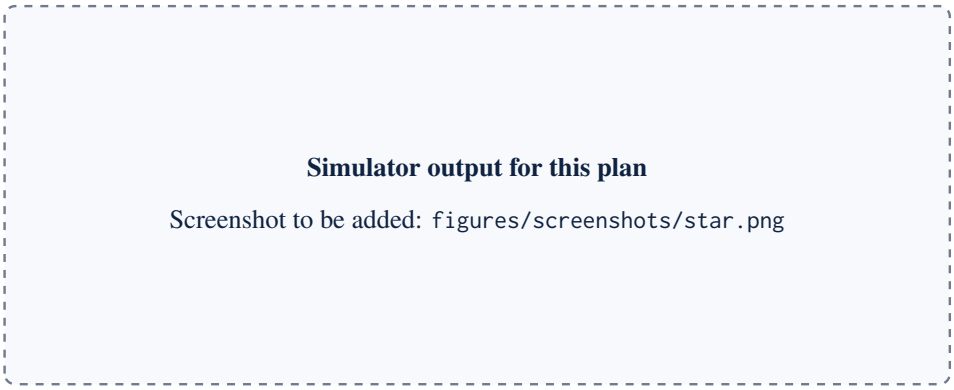


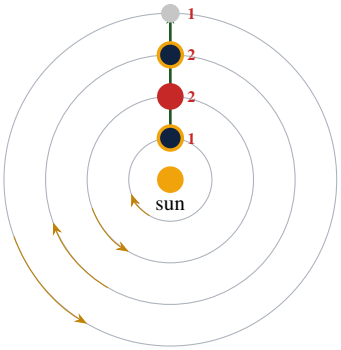
Figure 5.20: Star drone show in the simulator: the drones in the star shape round the picked centre.

5.9 Planetary Orbit Show

What the mission does. A solar-system show round one picked point (the sun), with one drone per planet. Drone k orbits at a radius of $50(k + 1)$ m, and neighbouring orbits turn in opposite directions (Figure 5.22b). One lap is one turn of the outermost drone; inner drones make as many whole turns as fit, so all drones line up again on the north line at the end of every lap.

At a glance: planetary orbit show	
Map input	one centre point (the sun); number of laps
Drones	2 to 8 (one per planet)
Neighbours per drone	1 or 2 – the drones on the next orbit in and the next orbit out
ENHSP plans	entering the orbits and the laps
Special feature	neighbouring orbits must turn in opposite directions

Communication. Each drone talks to the drones on the orbits just inside and just outside its own. The innermost and outermost drones therefore have one neighbour, and the others two (Figure 5.21).



Innermost and outermost drone: 1 neighbour.
Every other drone: 2 (next orbit in and next orbit out).
Neighbouring orbits turn in opposite directions.

Figure 5.21: Communication in the planetary show: each drone talks to the drones on the neighbouring orbits.

In PDDL this is the `orbit-neighbor` relation. The drones may enter their orbits only when every drone is up and in contact, and only if every pair of orbit neighbours turns in opposite directions:

```
;; problem: the two template planets are orbit neighbours
(orbit-neighbor drone-mercury drone-venus)
(orbit-neighbor drone-venus drone-mercury)
(neighbor-comm-ok drone-mercury)
(neighbor-comm-ok drone-venus)

;; domain: enter the orbits only if everyone is in contact and neighbours turn opposite ways
(:action enter-orbits
    :parameters (?lead - drone)
    :precondition (and (formation-leader ?lead) (at-cruise-altitude ?lead)
                       (neighbor-comm-ok ?lead) (not (orbits-entered))
                       (forall (?m - drone)
                           (or (not (has-orbit ?m))
                               (and (at-cruise-altitude ?m) (neighbor-comm-ok ?m))))
                       (forall (?a - drone ?b - drone)
                           (or (not (orbit-neighbor ?a ?b))
                               (and (orbits-clockwise ?a) (orbits-anticlockwise ?b))
                               (and (orbits-anticlockwise ?a) (orbits-clockwise ?b)))))
    :effect (orbits-entered))
```

Important PDDL. After the orbits are entered, one `fly-lap` action follows per lap, counted by `laps-completed`, and then `land-planets`:

```
(:action fly-lap
    :parameters (?lead - drone)
    :precondition (and (formation-leader ?lead) (orbits-entered)
                       (< (laps-completed) (laps-required))
                       (gps-ok ?lead) (communication-ok ?lead))
    :effect (increase (laps-completed) 1))
```

Action order. ENHSP applies the actions in this order:

```
pre-flight-check, takeoff, climb (each drone) → enter-orbits → fly-lap (once per lap) →
land-planets
```



(a) star show, 10 drones (b) planets, alternating directions

Figure 5.22: Shape-changing shows. Both are flown against a shared timetable so that every phase starts and ends together for all drones.

Simulator output for this plan

Screenshot to be added: `figures/screenshots/planets.png`

Figure 5.23: Planetary orbit show in the simulator: one drone per orbit round the picked centre.

5.10 Road Inspection

What the mission does. The operator clicks a source and a destination on a road. One drone takes off at the source, follows the road's exact path to the destination, flies the same road back and lands at the source. The road geometry comes from an OSRM routing server on OpenStreetMap data. If no road route exists, a click is more than 30 m from any road, or the two points are less than 20 m apart along the road, the plan fails before anything flies.

At a glance: road inspection	
Map input	two points on a road
Drones	1
Neighbours per drone	none – ground station link only
ENHSP plans	every road segment, out and back
Special feature	road geometry from OSRM; at most 12 segments

Communication. There is only one drone, so it has no neighbours. It keeps its link to the ground station during the whole inspection (Figure 5.24).

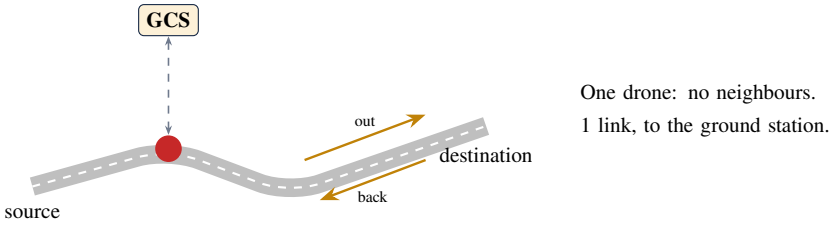


Figure 5.24: Communication in road inspection: a single drone linked to the ground station.

As in travel, the ground link is the flag `communication-ok`, which every inspection step requires.

Important PDDL. The road is split into at most 12 segments, each with its own length and energy. Each segment is inspected in order, and only if the battery covers it:

```
(:action inspect-segment
    :parameters (?d - drone ?from - location ?to - location)
    :precondition (and (inspector ?d) (road-segment ?from ?to)
                       (not (segment-inspected ?from ?to)) (not (road-inspected))
                       (at ?d ?from) (at-cruise-altitude ?d)
                       (gps-ok ?d) (communication-ok ?d) (drone-healthy ?d)
                       (> (battery ?d) (energy-required ?from ?to)))
    :effect (and (not (at ?d ?from)) (at ?d ?to) (segment-inspected ?from ?to)
                 (increase (segments-inspected) 1)
                 (decrease (battery ?d) (energy-required ?from ?to))))
```

Action order. ENHSP applies the actions in this order:

```
pre-flight-check → takeoff → climb-to-cruise-altitude → inspect-segment (each segment in order)
→ finish-inspection → return-along-road (each segment back) → land-at-source
```

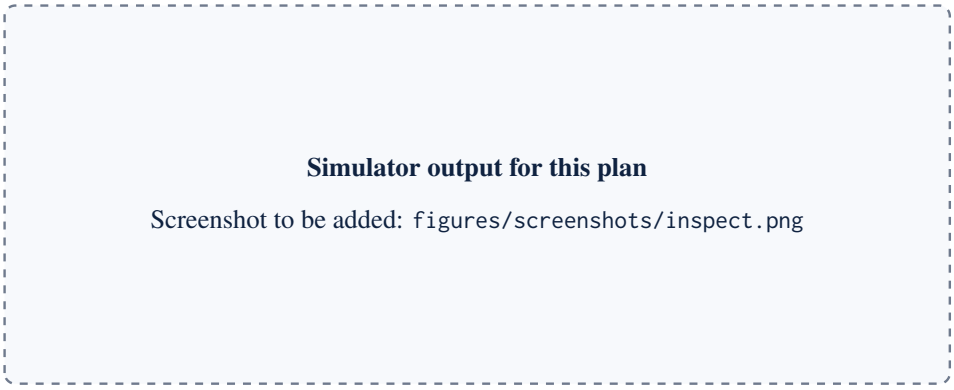


Figure 5.25: Road inspection in the simulator: the drone's route following the real road between the two clicked points.

5.11 Summary of the Plans

Table 5.1 compares the ten plans side by side.

Table 5.1: Comparison of the mission plans

Plan	Drones	Neighbours per drone	What ENHSP plans	Special feature
Travel	1–60	none	every drone's legs	full controller fault responses
Search	2–7	none	one lane per drone	pairwise separation check
V	3–9, odd	1–2	leader corridor	timed wing launch
Grid	4–36	2–4	leader corridor	4-neighbour links
Cube	8–64	up to 24	leader corridor	layer-by-layer ascent
Parabola	3	2	vertex corridor	range-checked link mesh, shape check
Figure-8	2–capacity	1–2	leader's lap	capacity rule, laps
Star	6–40	2	show phases	circle/star/spin sequence
Planets	2–8	1–2	show phases	opposite-direction orbits
Inspect	1	none	road segments	OSRM road geometry

Chapter 6

Inter-Drone Communication

Communication appears at two levels in the simulator. In the PDDL models it is a set of conditions: a drone may only act if its links are up. At run time it is a real exchange of messages between the ground station and the drones. This chapter explains both and how they relate. The communication of each individual plan, with its own diagram and PDDL, is given in the section of that plan in Chapter 5.

6.1 Communication in the PDDL Models

Two flags carry communication state for every drone:

- `communication-ok`: the link between the drone and the ground controller works;
- `neighbor-comm-ok`: the drone's links to its formation neighbours work.

Every action that moves a drone or changes the formation checks the relevant flags. A formation cannot be established, cruise a leg or close up if any member has lost its neighbour links. Each plan also declares *who* must talk to whom, using a neighbour relation suited to its shape (Table 6.1, Figure 6.1).

Table 6.1: Neighbour relations declared by each formation

Plan	Relation	**Who talks to whom**
V formation	`wing-neighbor`	leader with rank 1, then rank to rank along each wing (a chain)
Grid	`grid-neighbor`	up to 4 neighbours: front, back, left, right
Cube	`cube-neighbor`	up to 24 neighbours in the surrounding $3 \times 3 \times 3$ block, excluding directly above and below
Parabola	`parabola-neighbor`	all three pairs, in both directions (6 directed links)
Figure-8	`train-neighbor`	the drone in front and the drone behind
Star	`ring-neighbor`	the two drones on either side round the circle
Planets	`orbit-neighbor`	drones on adjacent orbits



(a) chain V wings, train — (b) 4-neighbour grid — (c) full mesh parabola — (d) ground link (all plans)

Figure 6.1: Communication topologies. (a)–(c) are drone-to-drone relations declared in PDDL; (d) is the ground-controller link represented by `communication-ok`.

6.2 The Range-Checked Link Model of the Parabola Plan

In most plans the links are summarised by the per-drone flag. The parabola plan models each directed link explicitly (Section 5.6). A link from d_1 to d_2 can be established only when both drones are airborne at cruise altitude, both radios work, and the straight-line distance between them is within the communication range of 100 m. A drone becomes `swarm-comm-ready` through `confirm-swarm-comm` once it has a link to every one of its neighbours. Only then can it take its slot. During flight, `hold-parabola-shape` requires every declared link to still be up, and if a drone loses its neighbour links, `detect-neighbor-link-loss` removes all links to and from that drone, so the shape cannot be re-formed until they are restored. A negative test with the range reduced to 15 m (below the 20 m arm-to-arm distance) correctly produced no plan. In the solved plan, the six `establish-comm-link` actions and three `confirm-swarm-comm` actions appear right after the climbs and before any slot is taken (Section 8.6).

6.3 Run-Time Communication between Ground Station and Drones

At run time each drone is an independent thread that owns its state. It can only be reached through a command inbox and only reports through a telemetry callback. Over the network, the ground station and drones use a simple JSON-over-UDP protocol, one datagram per message, designed so that it can later be replaced by MAVLink. Every message carries the drone's MAVLink SYSID, so one ground socket can address the whole swarm (Figure 6.2).

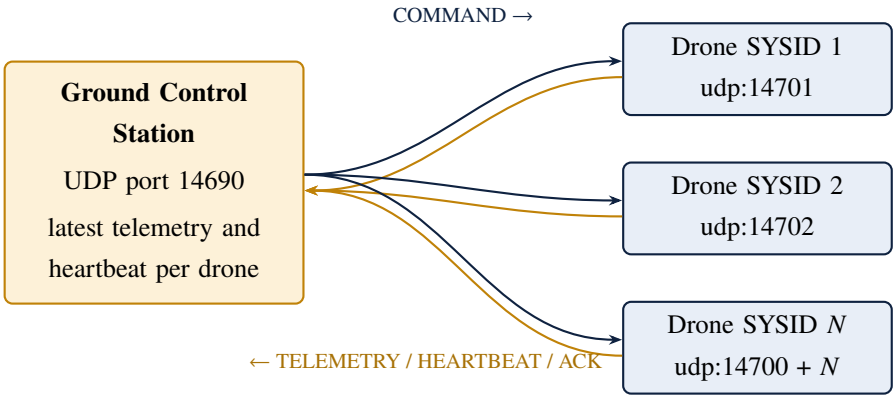


Figure 6.2: Run-time messaging. Commands are routed to a port derived from each drone's SYSID; every drone reports back to one ground station socket.

Table 6.2 lists the commands. A drone sends a heartbeat every second and telemetry every 0.5 s. The ground station treats a drone as out of contact if no heartbeat arrives for 3 s. On the drone side, if nothing is heard from the ground for more than 5 s the drone marks its link `DEGRADED`, and after 10 s `LOST`. Link loss is therefore *observed* from missing messages rather than assumed.

Table 6.2: Commands of the ground-station-to-drone protocol

Command	**Purpose**
`NAVIGATE`	The one mission operation: upload a whole route (approach, loop, laps, depart).
`ARM, DISARM, TAKEOFF, LAND`	Basic flight control.
`HOLD, RESUME`	Pause and continue the mission (used while the operator decides on a fault).
`RETURN_TO_LAUNCH, ABORT`	Fall back to the start point, or stop.
`SET_GPS`	Fault injection: switch the drone's GPS fix off or on.
`SET_SPEED`	Change cruise speed in flight.
`TRAIN_PACE`	Formation keeping: the shared, time-stamped pace of the train or show clock.
`PING, SHUTDOWN`	Liveness check and stop the drone thread.

6.4 On-Board Formation Keeping

The problem, in simple words. In the figure-eight train every drone must stay 10 m behind the drone in front. In the first version, the ground station watched all the drones and sent each one "speed up" or "slow down" commands. These commands always arrived a little late, two or three telemetry updates after the gap was measured. By then the gap had already changed. A drone told to speed up was still speeding up after it had caught up, then had to brake hard, and the drone behind copied the same mistake even more strongly. Like a traffic jam on a highway, the error grew along the train, and drones bunched together and even overtook each other.

The idea. The ground station no longer steers each drone. It acts like the conductor of a band who only keeps the beat. It broadcasts one shared message, `TRAIN_PACE`, which says where the head of the train should be, how fast it is moving and *when* this was measured. Each drone does the rest by itself, many times per second, using its own exact position.

What each drone does every tick.

1. **Bring the pace up to date.** The drone moves the pace forward by the time since its time stamp. So even a message that arrives late gives the correct position for now.
2. **Find its own target.** The drone subtracts its place in the train: drone number k should be $k \times 10\,\text{m}$ behind the pace point.
3. **Measure itself.** The drone measures how far it has really flown along its own route. This needs no message, so it has no delay.
4. **Adjust its own speed.** If it is behind its target it speeds up a little; if it is ahead it slows down a little.
5. **Stay safe.** If the drone in front is reported too close, the drone never speeds up, whatever its target says.

The ground station keeps one extra job: if any drone falls far behind, it slows the pace, so the whole train waits for that drone instead of leaving it behind.

Why it works. The only thing that comes from the ground is the pace, and the time stamp corrects for its delay. Everything else uses the drone's own position, which is never late. Figure 6.3 shows the loop.

The same idea in the shows. For the star and planets shows the same mechanism runs in *schedule* mode. Each drone gets a timetable of keyframes, "at clock time t, be s metres along your route", and the pace is simply a shared clock. Because every drone follows its own timetable against the same clock, every shape change starts and finishes at the same moment for all drones.

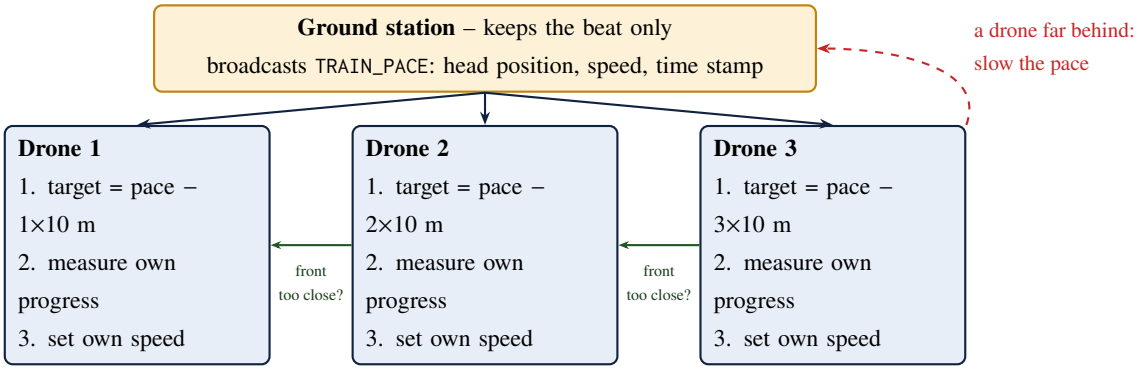


Figure 6.3: On-board formation keeping. The ground station only sends the pace; every drone works out its own target and sets its own speed, many times per second.

6.5 How the Two Levels Relate

The two levels play different roles, much like a travel plan and the driver who follows it. A travel plan is written before the trip, assuming that the roads are open. The driver is the one who actually sees a closed road on the way and stops to ask what to do. In the same way:

- **Before flight (PDDL planning).** The flags `communication-ok` and `neighbor-comm-ok` say that the links are assumed to work. The plan is valid only under this assumption.
- **During flight (run-time protocol).** Heartbeats, telemetry and `TRAIN_PACE` messages actually carry the commands and reports, and missing messages show that a link has really failed.
- **When they disagree.** If the run time observes a fault (Chapter 7), the mission is held and the operator decides how to continue. The planner is not called again.

Figure 6.4 summarises this. The PDDL conditions do not by themselves monitor real radio links; this is discussed as a limitation in Chapter 9.

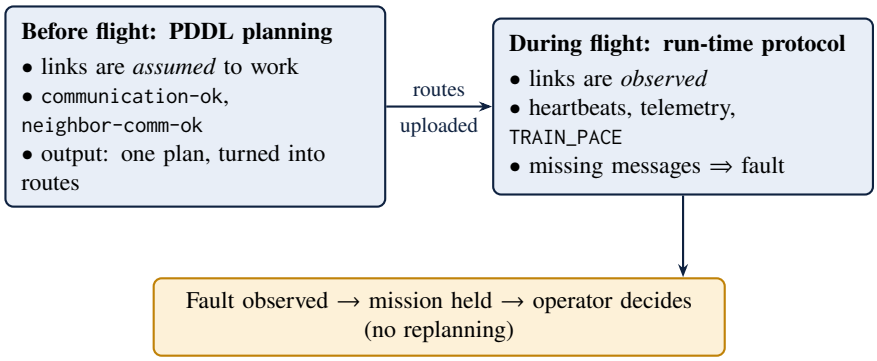


Figure 6.4: How planning-time and run-time communication relate.

Chapter 7

Fault Modelling and Battery Health

7.1 Faults in Simple Words

A *fault* is anything that stops a drone, or the whole swarm, from carrying out the mission as planned. Some faults are found *before* take-off, when the planner refuses to make a plan. Others happen *during* flight and are handled by the ground station and the operator. Table 7.1 lists the faults handled in this work, in plain words.

Table 7.1: Faults handled in this work, in simple words

Fault	What it means	What happens
Found before take-off (the planner refuses)		
Not enough battery	The battery cannot cover a leg and still keep the return reserve.	No plan uses that leg; if no other leg works, no plan at all.
No safe route	The no-fly area blocks every way from source to destination.	No plan; the operator must change the points or the area.
Too many drones	The train is too long for the drawn figure-eight.	No plan; the GUI warns before planning.
Wrong number of drones	For example an even number for a V, or not exactly three for a parabola.	The request is rejected before planning.
Unhealthy drone	GPS, radio or health already failed before take-off.	The drone is refused by the pre-flight check.
Found during flight (the ground station reacts)		
GPS lost	The drone no longer knows where it is.	The swarm is held; the operator chooses to bring the drone back or wait for GPS.
Ground link lost	The drone stops hearing from, or talking to, the ground station.	Marked degraded after a few seconds without messages, then lost.
Neighbour link lost	The drone can no longer talk to its formation neighbours.	The formation cannot continue until the link is back.
Poor health	The drone's health score falls below the allowed minimum.	The drone leaves the formation and the controller is told.
Low battery	The battery falls to the low threshold during flight.	The whole swarm is paused and the operator decides: land now or press on.
Empty battery	The battery reaches zero in flight.	The drone is lost.

7.2 How the Planner Models a Fault

Every domain contains the same small family of *detection* actions. Each one does three things: it notices that a flying drone has a failed flag, it records the failure, and it takes the drone out of the flight and out of the formation. Table 7.2 lists them.

Table 7.2: Fault detection actions shared by the domains

Action	**Fires when**	**Result**
`detect-gps-loss`	a flying drone has no GPS	GPS marked lost; drone stops and leaves the formation
`detect-communication-loss`	a flying drone has no ground link	link marked lost; drone stops and leaves the formation
`detect-neighbor-link-loss`	a flying drone cannot reach its neighbours	link marked lost; drone leaves the formation (parabola also drops all its links)
`detect-health-failure`	health falls below the minimum	health failure marked; drone stops and leaves the formation
`notify-ground-controller`	any failure has been marked	the ground controller is informed

For example, GPS loss is detected like this:

```
(:action detect-gps-loss
    :parameters (?d - drone)
    :precondition (and (flying ?d) (not (gps-ok ?d)))
    :effect (and (gps-lost ?d) (not (flying ?d)) (not (in-formation ?d))))
```

In words: if a drone is flying and its GPS is not OK, mark its GPS as lost, stop it and remove it from the formation. Because every movement and formation action also needs the health flags (`gps-ok`, `communication-ok`, `drone-healthy` and, in formations, `neighbor-comm-ok`), a drone with a failed flag simply cannot be moved by any plan. The planner therefore never flies an unhealthy drone, and a failed member stops the formation actions that depend on it.

7.3 What the Ground Controller Can Order

In the point-to-point domain, once the controller has been told about a failure it can choose one of three orders (Figure 7.1):

- **Return**: the drone flies back to the source over a safe route, if its battery covers the way back (`controller-order-return`, `return-to-source`).
- **Hover**: the drone stays where it is. Hovering uses battery, so when the battery reaches the return reserve the drone is switched to a return (`controller-order-hover`, `hover`, `initiate-low-battery-return`).

- **Sacrifice**: the drone lands immediately where it is, because it cannot be recovered (`sacrifice-drone`).

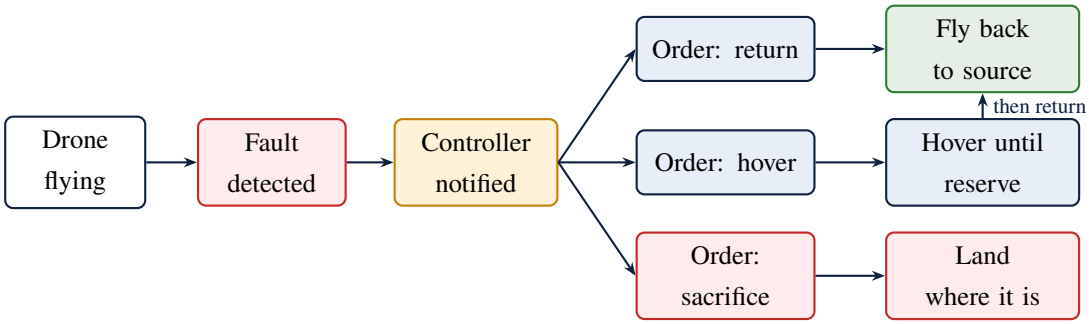


Figure 7.1: Ground-controller responses in the point-to-point domain: a detected fault is reported, and the controller orders a return, a hover or a sacrifice.

7.4 Battery as a Planning Resource

Battery is a number in every domain, with 100 meaning a full charge. The planner uses it in three simple ways.

Every leg costs energy. Each leg has an `energy-required` value. When a template is retargeted, this value is recomputed from the new leg length at the leg's own energy-per-metre rate, with a default of 0.0125 units per metre (12.5 units per kilometre). A drone may fly a leg only if its battery is greater than the cost, and the cost is then taken away.

Climbing, following and hovering cost energy too. Formation plans charge `climb-energy` for the climb to cruise altitude (in the parabola plan 4 units for the vertex and 5 for the arms, which climb 10 m further), and `cruise-leg-energy` (12 units) for each follower leg. The travel domain charges `hover-energy` for each hover step.

Always keep enough to get home. Every drone keeps a `minimum-return-battery` of 30 units. The pre-flight check refuses a drone whose battery is not above this reserve, every leg is checked as affordable or not, and the hover and search domains switch to a return when the battery reaches the reserve. Health is a separate score, so a drone can be refused for poor health even with a full battery.

7.5 Battery Health at Run Time

During flight each drone thread tracks its own battery. With the default settings a drone has a 5 200 mAh battery and draws a nominal 20 A in flight; its percentage is computed from the charge used. The drone classifies its own health from the battery and the GPS fix (Figure 7.2): `NOMINAL` above 20 %, `DEGRADED` at or below 20 % or without GPS, `CRITICAL` at or below 5 %, and `FAILED` if the battery runs out in flight, in which case the drone is lost.

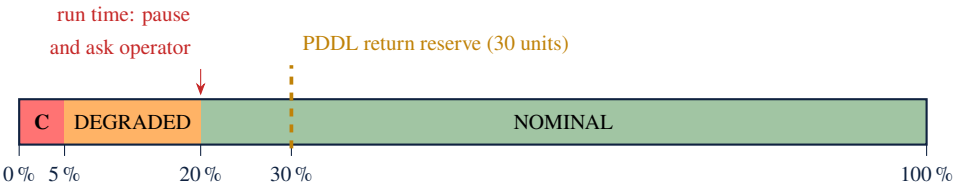


Figure 7.2: Battery thresholds. Below the bar: run-time health classes (C = CRITICAL; FAILED at 0 % in flight). The planner keeps a 30-unit reserve; at 20 % the run time holds the swarm and asks the operator.

When any drone reaches the low-battery threshold, the ground station *pauses the whole mission* so that the drones do not drain while the operator decides; at the simulator's compressed time scale the decision window would otherwise be a fraction of a second. A dialog shows the drone's battery and the nearest landing point, with a note whether it is within reach on the remaining charge. The operator may divert and land now, or press on, in which case the drone will be lost when the battery runs flat. A drone whose battery reaches zero in flight is reported as lost.

7.6 Fault Injection and Operator Decisions

The GUI's fault injection panel lets the operator pick one or more drones by SYSID and give them a fault (GPS loss, communications loss or battery failure), a severity and a duration (zero meaning until reset). In the thread-based backend, GPS loss is delivered as a `SET_GPS` command, so the drone itself loses its fix. On GPS loss the ground station holds the swarm and asks the operator whether the drone should fall back to its start point and land, showing the distance back and whether the remaining charge covers it; otherwise the drone holds position until GPS returns, while its battery keeps draining. When GPS comes back the mission resumes. Table 7.3 puts the planning side and the flight side of each fault next to each other.

Table 7.3: Each fault at planning time and during flight

Fault	In the plan (PDDL)	During flight
GPS loss	`gps-ok` false; `detect-gps-loss`	injected with `SET_GPS`; swarm held; operator chooses fall-back or hold
Ground link loss	`communication-ok` false; `detect-communication-loss`	noticed from missing heartbeats (degraded after 5 s, lost after 10 s)
Neighbour link loss	`neighbor-comm-ok` false; `detect-neighbor-link-loss`	formation keeping holds back (front-drone cap, slower pace)
Poor health	`health` below `minimum-health`; `detect-health-failure`	drone health classes `NOMINAL`, `DEGRADED`, `CRITICAL`, `FAILED`
Low battery	battery not above the reserve; leg refused, or return started	at 20 % swarm paused, operator chooses landing or press on; at 0 % drone lost

Chapter 8

Verification and Results

8.1 Method

Each plan was verified in two ways. First, ENHSP was run directly on problems produced by the real expand, retarget and solve pipeline, for a range of drone counts. Second, the resulting routes were flown headless through the drone threads, and the spacing, altitudes and landing positions were measured. Negative tests were added where a plan must refuse a problem. The run logs in `plans/*/runs/` record the in-app requests.

8.2 In-App Planning Requests

Table 8.1 summarises the logged requests for the four longest-running plans. Every unsolvable travel and V request was traced to an infeasible combination of source, destination and no-fly area, where no safe path existed. The grid failures all occurred in a short window before a template fix (Section 8.7); every grid request after the fix solved.

Table 8.1: Logged planning requests per plan

Plan	Runs	Solved	Unsolvable	Notes
Forest search	69	69	0	no failures on record
Travel	83	76	7	infeasible no-fly inputs during early testing
V formation	65	62	2	2 infeasible inputs; 1 run closed mid-plan
Grid	19	15	4	all 4 before the hover-altitude fix

8.3 Scalability of the Cube Formation

Table 8.2 and Figure 8.1 show planning time and plan length for the cube formation. Plan length grows linearly with the number of drones, about four actions per drone. Planning time stays below 4 s up to 36 drones and reaches 50 s for a full $4 \times 4 \times 4$ block of 64 drones. A launch-order audit of the 25-drone plan found no drone taking off before its layer opened and no layer opening before the one below had finished climbing.

Table 8.2: Cube formation: ENHSP results through the real pipeline

Drones	8	9	12	16	25	27	36	64
Shape	2×2×2	3×2×2	2×3×3	2×3×3	3×3×3	3×3×3	3×4×4	4×4×4
Time (s)	0.4	0.5	0.6	0.7	1.2	1.8	3.4	50.0
Plan length	42	47	58	74	111	119	155	268

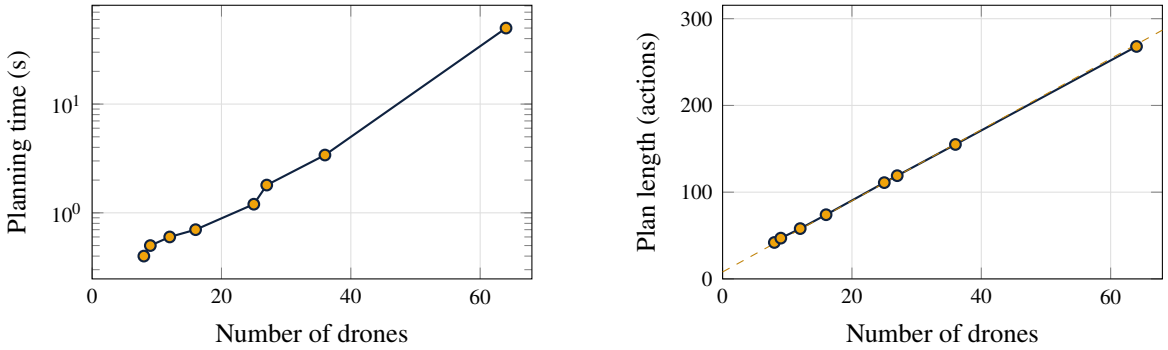


Figure 8.1: Cube formation scalability. Left: planning time (log scale). Right: plan length, close to linear in the number of drones (dashed line $\approx 4.1N + 8$).

8.4 Figure-Eight Capacity and Flight Quality

On an asymmetric eight with lobes of 337 m and 281 m, the computed capacity was 27 drones. ENHSP solved every count up to 27 and returned no plan for 28, in agreement with the Python check (Table 8.3). With a full 27-drone train, the minimum distance between any two drones at any waypoint step was 9.85 m, including at the crossing. Ten drones with 20 laps solved in 0.8 s, because laps are counted in PDDL and flown on board with a single repeat instruction rather than being unrolled.

Table 8.3: Figure-eight train: ENHSP results on an eight of capacity 27

Drones	2	5	12	25	27	28
Time (s)	0.4	0.4	0.7	2.7	3.3	–
Plan length	26	38	66	118	126	no plan (as intended)

After formation keeping moved on board (Section 6.4), a replay of a logged 16-drone, 2-lap run changed from a median neighbour gap of 23.7 m with a minimum of 0.0 m (collisions) to a median of 10.0 m, with 90 % of samples between 9.2 and 10.8 m, and every drone landed on its own pad.

8.5 Other Plans

Table 8.4 summarises the verification of the remaining plans.

Table 8.4: Verification summary of the remaining plans

Plan	Result
V formation	3-, 5- and 7-drone V's solve through the real pipeline after the neighbour chain was added.
Parabola	27-step plan with a detour, about 0.03–0.1 s. Negative checks (arm at 55 m, flat 50 m line, communication range 15 m) give no plan. In flight, altitudes held at 50/60/60 m, vertex–arm gap 14.1–15.0 m on a straight 1 km route.
Star show	6 drones × 1 lap 0.3 s; 40 drones × 5 laps 0.6 s. In flight (10 drones, 2 laps), worst drone 0.7–3.2 m off its slot while shapes were held, closest pair 9.3 m, every drone landed on its own pad.
Planets	2–8 drones × 3 laps, 0.2–0.3 s. With 8 drones every drone stayed within 0.4 m of its orbit and turned the correct direction and number of turns.
Road inspection	1.4 km and 3.8 km real roads followed with no measurable deviation from the road line; ENHSP 0.3 s; a source 393 m off the road was correctly refused.

8.6 A Complete Sample Plan

The parabola plan below, from a logged run with one detour location, shows the whole life cycle: pre-flight checks, take-offs and climbs, six link set-ups and three confirmations, slots, formation, two legs each followed by a shape hold, and completion.

ENHSP output — parabola formation with one detour

```
  0.0: (pre-flight-check drone-vertex source)
  1.0: (takeoff drone-vertex source)
  2.0: (climb-to-cruise-altitude drone-vertex)
  3.0: (pre-flight-check drone-left source)
  4.0: (takeoff drone-left source)
  5.0: (climb-to-cruise-altitude drone-left)
  6.0: (pre-flight-check drone-right source)
  7.0: (takeoff drone-right source)
  8.0: (climb-to-cruise-altitude drone-right)
  9.0: (establish-comm-link drone-vertex drone-right)
 10.0: (establish-comm-link drone-vertex drone-left)
 11.0: (confirm-swarm-comm drone-vertex)
 12.0: (establish-comm-link drone-left drone-vertex)
 13.0: (establish-comm-link drone-left drone-right)
 14.0: (establish-comm-link drone-right drone-left)
 15.0: (establish-comm-link drone-right drone-vertex)
 16.0: (confirm-swarm-comm drone-right)
 17.0: (confirm-swarm-comm drone-left)
 18.0: (take-parabola-slot drone-vertex)
 19.0: (take-parabola-slot drone-left)
 20.0: (take-parabola-slot drone-right)
 21.0: (establish-parabola-formation drone-vertex source)
 22.0: (formation-cruise drone-vertex source detour1)
 23.0: (hold-parabola-shape drone-vertex detour1)
 24.0: (formation-cruise drone-vertex detour1 destination)
 25.0: (hold-parabola-shape drone-vertex destination)
 26.0: (complete-parabola-mission drone-vertex destination)
Plan-Length: 27   Planning Time: 31 ms   Expanded Nodes: 28   States Evaluated: 77
```

8.7 Defects Found and Fixed

Verification found several modelling defects. They are recorded here because each one taught a general lesson about writing PDDL for a template-expanding pipeline.

Table 8.5: Modelling defects found during verification

Plan	Defect	Fix and lesson
Grid	`hover-altitude` facts were written over three lines. The line-by-line cloner copied only the middle line, leaving the value undefined for every added drone; AIBR then reported any grid larger than 1×2 as unsolvable.	Rewritten as one line per fact. Templates must keep one fact per line.
Cube	Layer gate written as a comparison of two fluents. A 25-drone, 3-layer cube did not solve in 90 s.	Replaced by propositional layer objects; solved in 1.2 s. Prefer propositional gates for ordering.
Cube	A quantified variable `?l` in `establish-cube-formation` shadowed the action's location parameter, making every problem unsolvable.	Variable renamed. Never reuse a parameter name inside a quantifier.
V formation	Readiness checked only the ground link; there was no notion of which drones sit next to each other.	Added `wing-neighbor` chain and `neighbor-comm-ok`, with `detect-neighbor-link-loss`.
Figure-8	Ground-station correction caused oscillation and collisions in a 16-drone train.	Formation keeping moved on board with a shared pace (Section 6.4).

Chapter 9

Discussion and Limitations

9.1 What the Models Guarantee

A plan returned by ENHSP is a proof, within the model, that the mission can be completed without breaking any of the written rules: no drone flies with a failed flag, no move exceeds the battery, no formation forms without all its members in contact, the cube never climbs a layer through another, and the figure-eight never carries more drones than it can hold. Because these rules are written in readable PDDL, they can be reviewed directly, and a change to a rule takes effect in every mission that uses it.

9.2 Limitations

The present work has the following limitations, in simple terms.

- **Flying at a constant height.** Each drone flies the whole mission at one planned cruise height. The planner does not change a drone's height along the route, for example to climb over a tall building or a hill, and it has no model of the terrain below.
- **No replanning when a plan fails.** The plan is made once, before take-off. If a drone fails or the route becomes blocked during the flight, the planner is not called again: the swarm is paused and the operator decides what to do.
- **No backup plans.** The fault detection actions record a failure, but the planner never prepares a "plan B" in advance, such as which drone should take over a lost drone's place.
- **Communication is assumed, not measured.** The PDDL flags say that the links are expected to work. They are not updated from real radio signal strength during the flight.
- **Battery in model units.** Battery and energy in PDDL use a 0–100 scale, not real milliampere-hours. The planning reserve and the run-time low-battery threshold are therefore two separate safeguards rather than one consistent model.
- **Only fixed, known no-fly areas.** One restricted area is drawn before planning. Moving obstacles, other aircraft, wind and weather are not modelled.

- **Followers are not checked one by one.** Only the leader's path is planned. The followers' paths are derived from it by fixed offsets, with a wider safety margin around the no-fly area, so the planner does not check each follower's path on its own.
- **Planning time grows for some formations.** The V formation becomes slow to plan as the swarm grows: beyond 9 drones it no longer finishes within the simulator's planning time limit, while the grid and the cube scale much further.
- **Partial fault injection.** The GUI offers GPS loss, communications loss and battery failure, but in the thread-based backend only GPS loss is currently delivered to the drones.
- **Simulation only.** All results come from the simulator. No physical flight was performed, and the road inspection plan depends on an external routing service.

Chapter 10

Conclusion and Future Work

10.1 Conclusion

This thesis presented the PDDL planning layer of a drone swarm simulator. Ten mission plans were built on one shared vocabulary, covering travel, area search, four rigid formations, a figure-eight train, two shape-changing shows and road inspection, and every plan adapts to the number of drones the operator selects. Communication was modelled as neighbour relations with per-drone flags, and, in the parabola plan, as an explicit mesh of range-checked links. Faults were modelled as detection actions with ground-controller responses, and battery as a numeric resource with a return reserve, complemented at run time by thresholds that pause the swarm and ask the operator.

The plans run through the real pipeline: they are retargeted to map inputs, expanded to the selected number of drones, solved by ENHSP and flown by the drone threads. The results show that careful modelling keeps planning fast for realistic swarm sizes: 64 drones in a cube in 50 s, a 27-drone figure-eight train in 3.3 s, and most missions in well under a second. The defects found during verification led to general modelling rules that will help anyone extending the plans.

10.2 Future Work

1. **Online replanning.** Feed run-time faults back into the PDDL state and replan the remaining mission, for example closing a formation around a lost drone.

2. **Calibrated energy model.** Derive `energy-required` and the reserve from the drone's battery capacity, current draw, speed and wind, so that planning and run-time thresholds use the same units.

3. **A local planner on every drone.** Give each drone its own small on-board planner, so that it can make quick local decisions by itself – for example moving around a sudden obstacle or closing the gap when a neighbour is lost – without waiting for the ground station. The PDDL plan from the ground would still set the overall mission, while each drone's local planner handles what happens around it.

4. **Complete fault injection.** Deliver communication loss and battery failure faults to the drone threads, matching the GUI options, and test the operator responses under each.

5. **Larger formations.** Reduce planning cost for formations that grow slowly, such as the V, for example by the same propositional restructuring that made the cube scale.

6. **Tests on real drones.** Fly the plans with hardware in the loop and, eventually, on physical drones.

References

[1] D. McDermott, M. Ghallab, A. Howe, C. Knoblock, A. Ram, M. Veloso, D. Weld and D. Wilkins, *PDDL – The Planning Domain Definition Language*, Technical Report CVC TR-98-003, Yale Center for Computational Vision and Control, 1998.

[2] M. Fox and D. Long, *PDDL2.1: An Extension to PDDL for Expressing Temporal Planning Domains*, Journal of Artificial Intelligence Research, vol. 20, pp. 61–124, 2003.

[3] M. Ghallab, D. Nau and P. Traverso, *Automated Planning: Theory and Practice*, Morgan Kaufmann, 2004.

[4] E. Scala, P. Haslum, S. Thiébaux and M. Ramirez, *Interval-Based Relaxation for General Numeric Planning*, Proceedings of the 22nd European Conference on Artificial Intelligence (ECAI), pp. 655–663, 2016.

[5] E. Scala, P. Haslum, S. Thiébaux and M. Ramirez, *Subgoaling Techniques for Satisficing and Optimal Numeric Planning*, Journal of Artificial Intelligence Research, vol. 68, pp. 691–752, 2020.

[6] S.-J. Chung, A. A. Paranjape, P. Dames, S. Shen and V. Kumar, *A Survey on Aerial Swarm Robotics*, IEEE Transactions on Robotics, vol. 34, no. 4, pp. 837–855, 2018.

[7] M. Brambilla, E. Ferrante, M. Birattari and M. Dorigo, *Swarm Robotics: A Review from the Swarm Engineering Perspective*, Swarm Intelligence, vol. 7, no. 1, pp. 1–41, 2013.

[8] C. W. Reynolds, *Flocks, Herds and Schools: A Distributed Behavioral Model*, Proceedings of SIGGRAPH '87, Computer Graphics, vol. 21, no. 4, pp. 25–34, 1987.

[9] L. E. Dubins, *On Curves of Minimal Length with a Constraint on Average Curvature, and with Prescribed Initial and Terminal Positions and Tangents*, American Journal of Mathematics, vol. 79, no. 3, pp. 497–516, 1957.

[10] D. Luxen and C. Vetter, *Real-Time Routing with OpenStreetMap Data*, Proceedings of the 19th ACM SIGSPATIAL International Conference on Advances in Geographic Information Systems, pp. 513–516, 2011.

[11] MAVLink Development Team, *MAVLink Micro Air Vehicle Communication Protocol*, `https://mavlink.io/en/`.

[12] E. Scala, *ENHSP – Expressive Numeric Heuristic Search Planner*, `https://sites.google.com/view/enhsp/`.

[13] The Qt Company, *Qt for Python (PySide6)*, `https://doc.qt.io/qtforpython-6/`.